

GUIA DIDÁTICO DE IMPLEMENTAÇÃO

PROJETO FINAL ARM64 + FPGA

Tang Nano 4K • QEMU • Assembly AArch64 • Verilog • HDMI • LEDs externos

Versão reestruturada para quem está começando do zero

Aluno: João Paulo Fonseca de Araújo

Data de revisão: 26 de setembro de 2026

1. O que foi corrigido nesta versão do guia

O guia original contém a arquitetura e os códigos necessários, mas mistura três tipos de arquivo (arquivos que você cria, arquivos oficiais que você copia e arquivos que as ferramentas geram). Além disso, ele apresenta duas organizações de pastas em momentos diferentes. Para uma pessoa que ainda não conhece FPGA, Gowin ou Assembly, isso torna difícil saber o que fazer primeiro.

Regra principal: neste guia existe UMA estrutura oficial de pastas. Sempre siga os caminhos desta versão.

- Todo arquivo é marcado mentalmente como CRIAR, COPIAR OFICIAL ou GERADO AUTOMATICAMENTE.
- Os arquivos oficiais da Sipeed ficam intactos; você não precisa reescrevê-los nem entender cada linha deles para concluir o projeto.
- O projeto físico do Gowin fica em `fpga/gowin/final_project.gprj`. Essa localização resolve a ambiguidade do guia anterior.
- As simulações usam `fpga/sim`. O Gowin NÃO usa os testbenches.
- `commands.bin`, `commands.hex`, arquivos `.vvp`, `.vcd` e o `bitstream.fs` são produtos das ferramentas; não devem ser digitados manualmente.
- Foi acrescentada uma etapa separada de bring-up dos LEDs para você testar a parte elétrica antes de misturá-la com HDMI, MAC e memória.

2. Antes de programar: a analogia da fábrica

Pense no projeto como uma pequena fábrica. O lado ARM64 é o escritório que escreve pedidos. A FPGA é o chão de fábrica que recebe os pedidos e executa ações. O HDMI é o painel grande na parede; os LEDs são luzes de status; a BSRAM é um caderno de histórico; e os testbenches são ensaios feitos antes de ligar a máquina real.

Parte	Analogia	Função real
<code>arm64/</code>	Escritório	Cria pacotes de 10 bytes no QEMU.
<code>bridge/</code>	Tradutor/fiscal	Converte binário em hexadecimal e confere CRC/pacotes.
<code>fpga/rtl/</code>	Máquinas da fábrica	Lógica Verilog que será sintetizada na Tang Nano 4K.
<code>fpga/sim/</code>	Laboratório de ensaio	Testa a lógica sem depender da placa.
<code>fpga/constraints/</code>	Mapa de tomadas	Diz qual sinal vai para qual pino físico e qual é o clock.
<code>fpga/gowin/</code>	Oficina do fabricante	Projeto do Gowin com HDMI oficial + seu <code>top_final</code> .
<code>evidence/</code>	Pasta de provas	Prints, waveforms, fotos, logs e relatórios para a entrega.

3. Vocabulário mínimo para não se perder

Termo	Explicação simples
-------	--------------------

FPGA	Chip reconfigurável. Você descreve circuitos com Verilog; não está executando um programa tradicional linha por linha.
RTL	Descrição do comportamento do hardware em nível de registradores. No seu projeto, são os arquivos .v em fpga/rtl.
Módulo	Uma peça de hardware descrita em Verilog. Um módulo pode usar outros módulos.
Top-level	O módulo principal. É o 'maestro' que conecta todas as peças. Aqui ele é top_final.
Clock	O tic-tac que sincroniza o circuito. A placa fornece 27 MHz; o projeto cria 126 MHz e 25,2 MHz.
PLL	Bloco que gera outro clock a partir do clock de entrada.
CLKDIV	Divisor de clock. Neste projeto, divide 126 MHz por 5 para obter aproximadamente 25,2 MHz.
Synthesis	Etapa que transforma Verilog em uma rede de recursos reais da FPGA.
Place and Route	Etapa que decide onde cada recurso fica fisicamente no chip e como será conectado.
CST	Arquivo de pinagem: liga nomes como ext_led[0] ao pino físico da placa.
SDC	Arquivo de temporização: informa ao Gowin a frequência do clock.
Bitstream .fs	Arquivo final usado para configurar a FPGA.
Testbench	Código de teste que simula entradas e verifica saídas. Não vai para a placa.
UART	Protocolo serial. Na sua entrega principal ele é exercitado dentro da co-simulação.
CRC	Número calculado a partir dos bytes para detectar pacote corrompido.
MAC	Multiply-Accumulate: multiplica dois valores e soma o produto ao acumulador.
Q8.8	Formato de ponto fixo de 16 bits: 8 bits inteiros e 8 fracionários. O valor 256 representa 1,0.
BSRAM/BRAM	Memória interna em blocos da FPGA.
SVO/TMDS/OSER10/ELVDS	Peças do pipeline HDMI. Trate esse conjunto como um bloco oficial já pronto e preserve os arquivos da Sipeed.

4. A estrutura DEFINITIVA de pastas

```
final_project/
├── README.md          [CRIAR]
├── Makefile           [CRIAR]
├── arm64/
│   ├── Makefile       [CRIAR]
│   └── src/
│       ├── main.s      [CRIAR]
│       ├── packet.s    [CRIAR]
│       └── syscalls.s   [CRIAR]
│   └── build/          [GERADO]
├── bridge/
│   ├── bin_to_hex.py   [CRIAR]
│   └── check_packets.py [CRIAR]
├── docs/
│   └── protocol.md      [CRIAR]
├── evidence/
│   ├── waveforms/      [VOCÊ SALVA]
│   ├── synthesis/       [VOCÊ SALVA]
│   ├── hardware/       [VOCÊ SALVA]
│   └── logs/           [VOCÊ SALVA]
├── fpga/
│   ├── bringup/
│   │   ├── led_bringup.v    [CRIAR]
│   │   └── led_bringup.cst  [CRIAR]
│   ├── constraints/
│   │   ├── final.cst       [CRIAR]
│   │   └── final.sdc       [CRIAR]
│   ├── rtl/
│   │   ├── top_final.v     [CRIAR]
│   │   ├── protocol_system.v [CRIAR - simulação]
│   │   ├── common/
│   │   │   ├── reset_sync.v [CRIAR]
│   │   │   └── sync2ff.v    [CRIAR]
│   │   ├── control/
│   │   │   ├── command_executor.v [CRIAR]
│   │   │   ├── packet_parser.v    [CRIAR]
│   │   │   ├── response_builder.v [CRIAR]
│   │   │   ├── uart_rx.v          [CRIAR]
│   │   │   └── uart_tx.v          [CRIAR]
│   │   ├── dsp/
│   │   │   └── mac_unit.v         [CRIAR]
│   │   ├── io/
│   │   │   ├── debounce_pulse.v  [CRIAR]
│   │   │   ├── button_commands.v [CRIAR]
│   │   │   └── led_status.v       [CRIAR]
│   │   ├── memory/
│   │   │   └── telemetry_bram.v    [CRIAR]
│   │   └── video/
│   │       ├── pattern_stream.v    [CRIAR]
│   │       └── svo_hdmi_final.v     [CRIAR]
│   ├── sim/
│   │   ├── Makefile             [CRIAR]
│   │   ├── tb_mac_unit.v        [CRIAR]
│   │   ├── tb_led_status.v      [CRIAR]
│   │   ├── tb_pattern_stream.v  [CRIAR]
│   │   ├── tb_arm_fpga_protocol.v [CRIAR]
│   │   ├── commands.hex         [GERADO]
│   │   └── build/               [GERADO]
│   ├── third_party/
│   │   └── TangNano-4K-example/   [CLONAR - NÃO EDITAR]
│   ├── gowin/
│   │   ├── final_project.gprj    [CRIADO PELO SAVE AS]
│   │   ├── src/...               [COPIADO DO hdmi_720p OFICIAL]
│   │   ├── impl/...              [GERADO PELO GOWIN]
│   │   └── release/              [COPIAR .fs APROVADO AQUI]
```

Muito importante: não crie `commands.hex`, `commands.bin`, `build/`, `impl/` nem o `.fs` na mão. Eles aparecem quando os comandos corretos são executados.

5. Windows e WSL: o mesmo lugar com dois nomes

O Gowin roda no Windows. Os comandos de Assembly, Python e Icarus rodam no Ubuntu/WSL. A pasta é a mesma; só muda a forma de escrever o caminho.

Ambiente	Caminho
Windows / Gowin	C:\Projetos\vga-framebuffer-arm-fpga\final_project
WSL / Ubuntu	/mnt/c/Projetos/vga-framebuffer-arm-fpga/ final_project

Analogia: é a mesma casa vista por duas ruas diferentes. Não mantenha duas cópias do projeto.

6. Preparação do computador

No PowerShell, localize ou clone seu repositório:

```
New-Item -ItemType Directory -Force C:\Projetos | Out-Null
Set-Location C:\Projetos
if (-not (Test-Path .\vga-framebuffer-arm-fpga)) {
    git clone https://github.com/jotaraujo/vga-framebuffer-arm-fpga.git
}
Set-Location .\vga-framebuffer-arm-fpga
```

No Ubuntu/WSL, instale as ferramentas e confira uma por uma:

```
sudo apt update
sudo apt install -y make binutils-aarch64-linux-gnu qemu-user python3 iverilog gtkwave git

aarch64-linux-gnu-as --version
qemu-aarch64 --version
iverilog -V
python3 --version
make --version
```

Só avance quando: todos os cinco comandos de versão responderem sem 'command not found'.

7. Crie as pastas sem improvisar nomes

```
cd /mnt/c/Projetos/vga-framebuffer-arm-fpga

mkdir -p final_project/arm64/src
mkdir -p final_project/bridge
mkdir -p final_project/docs
mkdir -p final_project/evidence/{waveforms,synthesis,hardware,logs}
```

```
mkdir -p final_project/fpga/rtl/{common,control,dsp,io,memory,video}
mkdir -p final_project/fpga/{sim,constraints,bringup,third_party,gowin,release}
```

Confira a árvore com:

```
find final_project -maxdepth 4 -type d | sort
```

8. A base HDMI oficial: o que copiar e onde colocar

Aqui está uma das partes que mais confundem no guia original. Você NÃO deve pegar arquivos SVO, PLL e CLKDIV um por um e espalhá-los no seu rtl. A estratégia mais segura é guardar o repositório oficial inteiro em third_party e fazer uma cópia de trabalho da pasta hdmi_720p para fpga/gowin.

1. Entre em final_project/fpga/third_party.
2. Clone o repositório oficial da Sipeed.
3. Copie a pasta hdmi_720p inteira para ../gowin.
4. Nunca edite a cópia que está em third_party; ela serve como referência limpa.
5. Abra e modifique somente a cópia em fpga/gowin.

```
cd /mnt/c/Projetos/vga-framebuffer-arm-fpga/final_project/fpga/third_party
git clone --depth 1 https://github.com/sipeed/TangNano-4K-example.git
cp -r TangNano-4K-example/hdmi_720p ../gowin

test -f ../gowin/hdmi.gprj && echo "OK: hdmi.gprj encontrado"
```

A cópia de trabalho precisa conter, no mínimo, estes arquivos do projeto oficial:

```
fpga/gowin/
├─ hdmi.gprj
├─ src/
│   ├── gowin_clkdiv/gowin_clkdiv.v
│   ├── gowin_pllvr/gowin_pllvr.v
│   └─ hdmi/
│       ├── svo_defines.vh
│       ├── svo_enc.v
│       ├── svo_openldi.v
│       ├── svo_tcard.v
│       ├── svo_term.v
│       ├── svo_tmds.v
│       ├── svo_utils.v
│       └─ svo_vdma.v
├─ svo_hdmi.v
├─ top.v
├─ hdmi.cst
└─ hdmi.sdc
```

Arquivo oficial	O que você faz
gowin_pllvr.v	Mantém no projeto. Não reescreve. Gera o clock rápido.
gowin_clkdiv.v	Mantém no projeto. Não reescreve. Gera o clock de pixel.
src/hdmi/*.v e *.vh	Mantém a pasta inteira. São os blocos SVO/TMDS.
src/svo_hdmi.v	Use como referência. Seu projeto usa

	rtl/video/svo_hdmi_final.v.
src/top.v	Pode permanecer no disco/projeto, mas NÃO é o top final.
src/hdmi.cst e src/hdmi.sdc	Use primeiro para testar o exemplo oficial; depois remova do projeto ativo e substitua por final.cst/final.sdc.

Por que não colar o código oficial neste guia?: porque esses arquivos pertencem ao projeto externo da Sipeed e já vêm prontos no repositório. Copiar a pasta inteira preserva licenças, includes e versões dos IPs.

9. Primeiro teste: HDMI oficial sem seu código

6. Abra o Gowin EDA no Windows.
7. File > Open: C:\Projetos\vga-framebuffer-arm-fpga\final_project\fpga\gowin\hdmi.gprj.
8. Confirme o dispositivo GW1NSR-LV4CQN48PC6/I5.
9. Sem adicionar nenhum arquivo seu, execute Synthesize e depois Place and Route.
10. Programe a placa em SRAM com o .fs gerado.
11. Ligue a saída HDMI da Tang Nano à ENTRADA HDMI do monitor.
12. Se o padrão oficial não aparecer estável, pare aqui. O problema ainda não é o seu MAC, parser ou botões.

Analogia: antes de reformar uma casa, teste se água e energia já funcionam. HDMI oficial é esse teste básico.

10. Segundo teste: três LEDs isolados

Antes de misturar LEDs com HDMI, teste os três GPIO sozinhos. Isso elimina dúvidas de resistor, polaridade e pino.

Crie fpga/bringup/led_bringup.v:

```
module led_bringup (
    input wire clk,
    output reg [2:0] ext_led
);
    reg [24:0] count;
    reg [5:0] por_count = 6'd0;
    wire rst_n = &por_count;

    always @(posedge clk) begin
        if (!rst_n)
            por_count <= por_count + 1'b1;
    end

    always @(posedge clk) begin
        if (!rst_n) begin
            count <= 25'd0;
            ext_led <= 3'b001;
        end else if (count == 25'd26_999_999) begin
            count <= 25'd0;
            case (ext_led)
                3'b001: ext_led <= 3'b010;
                3'b010: ext_led <= 3'b100;
                default: ext_led <= 3'b001;
            endcase
        end
    end
endmodule
```

```
end else begin
    count <= count + 1'b1;
end
end
endmodule
```

Crie fpga/bringup/led_bringup.cst:

```
IO_LOC "clk" 45;
IO_PORT "clk" IO_TYPE=LVC MOS33;

IO_LOC "ext_led[0]" 39;
IO_PORT "ext_led[0]" IO_TYPE=LVC MOS33 DRIVE=8 PULL_MODE=NONE;
IO_LOC "ext_led[1]" 40;
IO_PORT "ext_led[1]" IO_TYPE=LVC MOS33 DRIVE=8 PULL_MODE=NONE;
IO_LOC "ext_led[2]" 41;
IO_PORT "ext_led[2]" IO_TYPE=LVC MOS33 DRIVE=8 PULL_MODE=NONE;
```

Crie um projeto Gowin mínimo só para esse teste, selecione led_bringup como top e use apenas led_bringup.cst. O comportamento esperado é LED0 → LED1 → LED2, um por segundo.

Segurança: desligue USB-C/HDMI antes de mexer nos jumpers. Use um resistor por LED, preferencialmente 330 a 470 ohms.

11. Como montar a protoboard

13. Escolha um GND da Tang Nano e leve-o ao trilho negativo da protoboard.
14. GPIO físico 39 -> resistor -> ânodo do LED0; cátodo -> GND comum.
15. GPIO físico 40 -> resistor -> ânodo do LED1; cátodo -> GND comum.
16. GPIO físico 41 -> resistor -> ânodo do LED2; cátodo -> GND comum.
17. Não alimente o trilho da protoboard com 5 V nem 3,3 V. Cada LED recebe corrente pelo próprio GPIO.
18. Antes de ligar, confira se nenhum jumper encosta na linha vizinha e confirme os pinos no pin map da sua revisão.

12. Arquivos que VOCÊ cria: código completo e explicação

A partir daqui, faça exatamente um arquivo por vez. Salve, confira o caminho e só então vá para o próximo. Não cole Verilog no console do Gowin; o código deve existir como arquivo no disco.

12.1 arm64/src/main.s

Onde criar: final_project/arm64/src/main.s

Analogia: É a lista de pedidos do escritório. Ele percorre oito comandos e pede para packet.s embrulhar cada um.

Pergunta	Resposta
O que entra?	Tabela command_table embutida no próprio arquivo.
O que sai?	80 bytes em stdout: 8 pacotes × 10 bytes.
Com quem conversa?	Chama build_packet e write_all. O Makefile executa esse programa no QEMU.
Cuidado principal	stdout precisa conter apenas bytes. Qualquer texto de debug quebraria commands.bin.

Código completo:

```
.section .rodata
.align 3
command_table:
    // cmd, length, padding, payload little endian
    .byte 0x7F, 4
    .hword 0
    .word 0x44332211    // PING

    .byte 0x01, 1
    .hword 0
    .word 0x00000001    // SET_MODE 1

    .byte 0x02, 3
    .hword 0
    .word 0x00E08020    // SET_COLOR R=20 G=80 B=E0

    .byte 0x03, 2
    .hword 0
    .word 0x00000180    // SET_GAIN 1.5 em Q8.8

    .byte 0x04, 4
    .hword 0
    .word 0x00800100    // MAC_STEP 1.0 x 0.5

    .byte 0x04, 4
    .hword 0
    .word 0x0040FF00    // MAC_STEP -1.0 x 0.25

    .byte 0x01, 1
    .hword 0
    .word 0x00000003    // SET_MODE 3

    .byte 0x06, 0
    .hword 0
    .word 0x00000000    // GET_STATUS
command_table_end:

.section .bss
.align 4
```

```

packet_buffer:
    .skip 16

.section .text
.global _start
.type _start, %function
_start:
    adrp x20, command_table
    add x20, x20, :lo12:command_table
    adrp x21, command_table_end
    add x21, x21, :lo12:command_table_end
    mov w19, wzr          // sequencia começa em zero

.Lnext_command:
    cmp x20, x21
    b.hs .Lsuccess

    ldrb w2, [x20]        // command id
    ldrb w3, [x20, #1]    // payload length
    ldr w4, [x20, #4]     // quatro bytes do payload

    adrp x0, packet_buffer
    add x0, x0, :lo12:packet_buffer
    mov w1, w19
    bl build_packet

    adrp x0, packet_buffer

    add x0, x0, :lo12:packet_buffer
    mov x1, #10
    bl write_all
    cbnz w0, .Lwrite_error

    add x20, x20, #8
    add w19, w19, #1
    b .Lnext_command

.Lsuccess:
    mov x0, #0
    bl sys_exit

.Lwrite_error:
    mov x0, #1
    bl sys_exit

.size _start, .-_start

```

12.2 arm64/src/packet.s

Onde criar: final_project/arm64/src/packet.s

Analogia: É a máquina que coloca cada pedido dentro de um envelope padronizado e sela com CRC.

Pergunta	Resposta
O que entra?	sequência, ID de comando, length e payload.
O que sai?	pacote de 10 bytes no buffer.
Com quem conversa?	É chamado por main.s; usa crc8_update para calcular o byte 9.
Cuidado principal	CRC é calculado sobre os bytes 1 a 8, não sobre o SOF

	nem sobre o próprio CRC.
--	--------------------------

Código completo:

```
.section .text
.global build_packet
.type build_packet, %function
// x0 = buffer, w1 = sequence, w2 = command, w3 = length, w4 = payload
build_packet:
    stp x29, x30, [sp, #-48]!
    mov x29, sp
    stp x19, x20, [sp, #16]
    stp x21, x22, [sp, #32]

    mov x19, x0
    mov w22, w4

    mov w5, #0xA5
    strb w5, [x19, #0]
    mov w5, #0x01
    strb w5, [x19, #1]
    strb w1, [x19, #2]
    strb w2, [x19, #3]
    strb w3, [x19, #4]
    str w22, [x19, #5]

    mov w20, wzr        // CRC inicial 0x00
    mov x21, #1          // bytes 1 ate 8
.Lcrc_loop:
    ldrb w1, [x19, x21]
    mov w0, w20
    bl  crc8_update
    mov w20, w0
    add x21, x21, #1
    cmp x21, #9
    b.lo .Lcrc_loop

    strb w20, [x19, #9]
    mov x0, x19

    ldp x21, x22, [sp, #32]
    ldp x19, x20, [sp, #16]
    ldp x29, x30, [sp], #48
    ret
.size build_packet, .-build_packet

.global crc8_update
.type crc8_update, %function
// w0 = CRC anterior, w1 = novo byte; retorna w0 = CRC-8 polinomio 0x07
crc8_update:
    eor w0, w0, w1
    and w0, w0, #0xFF
    mov w2, #8

.Lcrc_bit:
    tst w0, #0x80
    lsl w0, w0, #1
    and w0, w0, #0xFF
    b.eq .Lcrc_no_xor
    eor w0, w0, #0x07
.Lcrc_no_xor:
    subs w2, w2, #1
    b.ne .Lcrc_bit
```

```
ret
.size crc8_update, .-crc8_update
```

12.3 arm64/src/syscalls.s

Onde criar: final_project/arm64/src/syscalls.s

Analogia: É o funcionário que leva o envelope até a saída do Linux e encerra o processo.

Pergunta	Resposta
O que entra?	ponteiro para buffer e quantidade de bytes.
O que sai?	write em stdout e exit code 0/1.
Com quem conversa?	main.s usa write_all e sys_exit.
Cuidado principal	Não depende de libc; por isso usa syscalls AArch64 diretamente.

Código completo:

```
.section .text
.global write_all
.type write_all, %function
// x0 = endereço, x1 = quantidade. Retorna w0 = 0 em sucesso, 1 em erro.
write_all:
    mov x3, x0
    mov x4, x1
.Lwrite_loop:
    cbz x4, .Lwrite_ok
    mov x0, #1          // stdout
    mov x1, x3
    mov x2, x4
    mov x8, #64         // write
    svc #0
    cmp x0, #0
    b.le .Lwrite_fail
    add x3, x3, x0
    sub x4, x4, x0
    b .Lwrite_loop
.Lwrite_ok:
    mov w0, wzr
    ret
.Lwrite_fail:
    mov w0, #1
    ret
.size write_all, .-write_all

.global sys_exit
.type sys_exit, %function
sys_exit:
    mov x8, #93         // exit
    svc #0
    b .
.size sys_exit, .-sys_exit
```

12.4 arm64/Makefile

Onde criar: final_project/arm64/Makefile

Analogia: É a receita automática do lado ARM64.

Pergunta	Resposta
O que entra?	arquivos .s em src.
O que sai?	objetos, packet_gen, commands.bin e commands.hex.
Com quem conversa?	chama os scripts de bridge depois de executar QEMU.
Cuidado principal	O caractere > funciona como prefixo de receita porque .RECIPEPREFIX foi definido.

Código completo:

```
CROSS ?= aarch64-linux-gnu
AS := $(CROSS)-as
LD := $(CROSS)-ld
QEMU ?= qemu-aarch64
.RECIPEPREFIX := >

BUILD := build
OBJS := $(BUILD)/main.o $(BUILD)/packet.o $(BUILD)/syscalls.o

.PHONY: all run hex check clean

all: $(BUILD)/packet_gen

$(BUILD):
>mkdir -p $(BUILD)

$(BUILD)/%.o: src/%.s | $(BUILD)
>$(AS) -g -o $$@ $$<

$(BUILD)/packet_gen: $(OBJS)
>$(LD) -static -e _start -o $$@ $(OBJS)

run: $(BUILD)/packet_gen
>$(QEMU) $(BUILD)/packet_gen > $(BUILD)/commands.bin

hex: run
>python3 ../bridge/bin_to_hex.py $(BUILD)/commands.bin ../fpga/sim/commands.hex

check: hex
>python3 ../bridge/check_packets.py $(BUILD)/commands.bin

clean:
>rm -rf $(BUILD)
```

12.5 bridge/bin_to_hex.py

Onde criar: final_project/bridge/bin_to_hex.py

Analogia: É um tradutor que escreve cada byte binário em uma linha hexadecimal legível pelo Verilog.

Pergunta	Resposta
O que entra?	arm64/build/commands.bin.
O que sai?	fpga/sim/commands.hex.
Com quem conversa?	O testbench carrega commands.hex com \$readmemh.

Cuidado principal	commands.hex não deve ser editado à mão.
-------------------	--

Código completo:

```
#!/usr/bin/env python3
from pathlib import Path
import sys

def main() -> int:
    if len(sys.argv) != 3:
        print("uso: bin_to_hex.py entrada.bin saida.hex", file=sys.stderr)
        return 2
    source = Path(sys.argv[1])
    target = Path(sys.argv[2])
    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_text(''.join(f'{byte:02x}\n' for byte in source.read_bytes()), encoding="ascii")
    print(f'{source}: {source.stat().st_size} bytes -> {target}')
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

12.6 bridge/check_packets.py

Onde criar: final_project/bridge/check_packets.py

Analogia: É o fiscal da expedição. Abre os pacotes e confere tamanho, SOF, versão e CRC.

Pergunta	Resposta
O que entra?	commands.bin.
O que sai?	linhas PASS/FALHA e código de saída.
Com quem conversa?	arm64/Makefile chama este script no alvo check.
Cuidado principal	Se retornar código 1, pare antes de simular a FPGA.

Código completo:

```
#!/usr/bin/env python3
from pathlib import Path
import sys

NAMES = {
    0x01: "SET_MODE",
    0x02: "SET_COLOR",
    0x03: "SET_GAIN",
    0x04: "MAC_STEP",
    0x05: "CLEAR",
    0x06: "GET_STATUS",
    0x7F: "PING",
}

def crc8(data: bytes) -> int:
    crc = 0
    for value in data:
        crc ^= value
    for _ in range(8):
        crc = ((crc << 1) ^ 0x07) & 0xFF if crc & 0x80 else (crc << 1) & 0xFF
```

```

return crc

def main() -> int:
    if len(sys.argv) != 2:
        print("uso: check_packets.py commands.bin", file=sys.stderr)
        return 2

    raw = Path(sys.argv[1]).read_bytes()
    if len(raw) == 0 or len(raw) % 10:
        print(f"FALHA: tamanho {len(raw)} nao e multiplo de 10", file=sys.stderr)
        return 1

    failed = 0
    for number in range(len(raw) // 10):
        packet = raw[number * 10:(number + 1) * 10]
        expected = crc8(packet[1:9])
        ok = packet[0] == 0xA5 and packet[1] == 1 and packet[9] == expected
        failed += not ok
        name = NAMES.get(packet[3], f"CMD_{packet[3]:02X}")
        print(
            f"pacote {number:02d} seq={packet[2]:02X} cmd={name:<10} "
            f"len={packet[4]} crc={packet[9]:02X} {'PASS' if ok else 'FALHA'}"
        )

    print(f"resultado: {len(raw) // 10 - failed} PASS, {failed} FALHA")
    return 1 if failed else 0

if __name__ == "__main__":
    raise SystemExit(main())

```

12.7 fpga/rtl/common/reset_sync.v

Onde criar: final_project/fpga/rtl/common/reset_sync.v

Analogia: É o porteiro: só libera o circuito quando o PLL realmente está pronto.

Pergunta	Resposta
O que entra?	clk e pll_lock em async_ready.
O que sai?	rst_n.
Com quem conversa?	top_final usa rst_n em praticamente todos os módulos.
Cuidado principal	reset ativo baixo: rst_n=0 mantém o circuito em reset.

Código completo:

```

module reset_sync (
    input wire clk,
    input wire async_ready,
    output wire rst_n
);
    reg [3:0] sync = 4'b0000;

    always @(posedge clk or negedge async_ready) begin
        if (!async_ready)
            sync <= 4'b0000;
        else
            sync <= {sync[2:0], 1'b1};
        end

    assign rst_n = sync[3];
endmodule

```

12.8 fpga/rtl/common/sync2ff.v

Onde criar: final_project/fpga/rtl/common/sync2ff.v

Analogia: É uma pequena antecâmara que traz um sinal externo para dentro do ritmo do clock.

Pergunta	Resposta
O que entra?	botão assíncrono.
O que sai?	versão sincronizada.
Com quem conversa?	fica antes do debounce.
Cuidado principal	não elimina bouncing; apenas reduz risco de metastabilidade.

Código completo:

```
module sync2ff #(
    parameter WIDTH = 1
)(
    input wire      clk,
    input wire      rst_n,
    input wire [WIDTH-1:0]  async_in,
    output reg [WIDTH-1:0]  sync_out
);
    reg [WIDTH-1:0] meta;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            meta    <= {WIDTH{1'b0}};
            sync_out <= {WIDTH{1'b0}};
        end else begin
            meta    <= async_in;
            sync_out <= meta;
        end
    end
endmodule
```

12.9 fpga/rtl/io/debounce_pulse.v

Onde criar: final_project/fpga/rtl/io/debounce_pulse.v

Analogia: É o filtro da campainha: uma apertada física não vira vários toques eletrônicos.

Pergunta	Resposta
O que entra?	button_in sincronizado.
O que sai?	button_level estável e pressed_pulse de um ciclo.
Com quem conversa?	top_final instancia um para KEY1 e outro para KEY2.
Cuidado principal	20 ms em 25,2 MHz é uma janela grande; não reduza sem motivo.

Código completo:

```
module debounce_pulse #()
```

```

parameter integer CLK_HZ = 25_200_000,
parameter integer DEBOUNCE_MS = 20
)(
    input wire clk,
    input wire rst_n,

    input wire button_in,
    output reg button_level,
    output reg pressed_pulse
);
localparam integer LIMIT = (CLK_HZ / 1000) * DEBOUNCE_MS;
localparam integer COUNT_W = $clog2(LIMIT + 1);
reg [COUNT_W-1:0] count;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        button_level <= 1'b0;
        pressed_pulse <= 1'b0;
        count <= {COUNT_W{1'b0}};
    end else begin
        pressed_pulse <= 1'b0;
        if (button_in == button_level) begin
            count <= {COUNT_W{1'b0}};
        end else if (count == LIMIT - 1) begin
            count <= {COUNT_W{1'b0}};
            button_level <= button_in;
            if (button_in)
                pressed_pulse <= 1'b1;
        end else begin
            count <= count + 1'b1;
        end
    end
end
endmodule

```

12.10 fpga/rtl/io/button_commands.v

Onde criar: final_project/fpga/rtl/io/button_commands.v

Analogia: É um intérprete: transforma botões nos mesmos comandos internos usados pelo protocolo.

Pergunta	Resposta
O que entra?	níveis/pulsos de KEY1 e KEY2.
O que sai?	cmd_valid, cmd_id e cmd_payload.
Com quem conversa?	alimenta command_executor no top físico.
Cuidado principal	KEY1 muda modo; KEY2 faz MAC_STEP; as duas juntas geram CLEAR.

Código completo:

```

module button_commands (
    input wire    clk,
    input wire    rst_n,
    input wire    key1_level,
    input wire    key2_level,
    input wire    key1_pulse,
    input wire    key2_pulse,
    input wire    cmd_ready,

```

```

output reg      cmd_valid,
output reg [7:0] cmd_id,
output reg [31:0] cmd_payload
);
localparam CMD_SET_MODE = 8'h01;
localparam CMD_MAC_STEP = 8'h04;
localparam CMD_CLEAR   = 8'h05;

reg [1:0] next_mode;
reg [2:0] sample_index;
reg      both_latched;

function signed [15:0] sample_value;
input [2:0] index;
begin
    case (index)
        3'd0: sample_value = 16'sd256;
        3'd1: sample_value = 16'sd128;
        3'd2: sample_value = -16'sd192;
        3'd3: sample_value = 16'sd384;
        3'd4: sample_value = -16'sd256;
        3'd5: sample_value = 16'sd64;
        3'd6: sample_value = 16'sd512;
        default: sample_value = -16'sd128;
    endcase
end
endfunction

function signed [15:0] coeff_value;
input [2:0] index;
begin

    case (index)
        3'd0: coeff_value = 16'sd128;
        3'd1: coeff_value = 16'sd256;
        3'd2: coeff_value = -16'sd64;
        3'd3: coeff_value = 16'sd192;
        3'd4: coeff_value = 16'sd320;
        3'd5: coeff_value = -16'sd256;
        3'd6: coeff_value = 16'sd64;
        default: coeff_value = 16'sd384;
    endcase
end
endfunction

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        next_mode <= 2'd0;
        sample_index <= 3'd0;
        both_latched <= 1'b0;
        cmd_valid <= 1'b0;
        cmd_id <= 8'd0;
        cmd_payload <= 32'd0;
    end else begin
        if (cmd_valid && cmd_ready)
            cmd_valid <= 1'b0;

        if (!(key1_level && key2_level))
            both_latched <= 1'b0;

        if (!cmd_valid) begin
            if (key1_level && key2_level && !both_latched) begin
                cmd_valid <= 1'b1;
                cmd_id <= CMD_CLEAR;
                cmd_payload <= 32'd0;
            end
        end
    end
end

```

```

        sample_index <= 3'd0;
        both_latched <= 1'b1;
    end else if (key1_pulse && !key2_level) begin
        cmd_valid <= 1'b1;
        cmd_id <= CMD_SET_MODE;
        cmd_payload <= {30'd0, next_mode + 2'd1};
        next_mode <= next_mode + 2'd1;
    end else if (key2_pulse && !key1_level) begin
        cmd_valid <= 1'b1;
        cmd_id <= CMD_MAC_STEP;
        cmd_payload <= {coeff_value(sample_index), sample_value(sample_index)};
        sample_index <= sample_index + 3'd1;
    end
end
end
end
endmodule

```

12.11 fpga/rtl/io/led_status.v

Onde criar: final_project/fpga/rtl/io/led_status.v

Analogia: É o painel de luzes da fábrica.

Pergunta	Resposta
O que entra?	modo ativo, comando aceito, erro, overflow e clear.
O que sai?	ext_led[2:0].
Com quem conversa?	LED0/1 mostram modo; LED2 mostra atividade/alerta.
Cuidado principal	o pulso é alongado para 200 ms; um pulso de um ciclo seria invisível.

Código completo:

```

module led_status #(
    parameter integer CLK_HZ = 25_200_000,
    parameter integer PULSE_MS = 200
)()
    input wire    clk,
    input wire    rst_n,
    input wire [1:0] active_mode,
    input wire    accepted_pulse,
    input wire    error_pulse,
    input wire    mac_overflow,
    input wire    clear_pulse,
    output wire [2:0] ext_led
);
    localparam integer PULSE_CYCLES = (CLK_HZ / 1000) * PULSE_MS;
    localparam integer COUNT_W = $clog2(PULSE_CYCLES + 1);
    reg [COUNT_W-1:0] pulse_count;
    reg alert_latched;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            pulse_count <= {COUNT_W{1'b0}};
            alert_latched <= 1'b0;
        end else begin
            if (clear_pulse)
                alert_latched <= 1'b0;
        end
    end
endmodule

```

```

    else if (error_pulse || mac_overflow)
        alert_latched <= 1'b1;

    if (accepted_pulse)
        pulse_count <= PULSE_CYCLES;
    else if (pulse_count != {COUNT_W{1'b0}})
        pulse_count <= pulse_count - 1'b1;
    end
end

assign ext_led[0] = active_mode[0];
assign ext_led[1] = active_mode[1];
assign ext_led[2] = alert_latched | (pulse_count != {COUNT_W{1'b0}});
endmodule

```

12.12 fpga/rtl/dsp/mac_unit.v

Onde criar: final_project/fpga/rtl/dsp/mac_unit.v

Analogia: É a calculadora principal: multiplica e acumula.

Pergunta	Resposta
O que entra?	dois operandos Q8.8, start e clear.
O que sai?	acumulador Q16.16, resultado Q8.8, done e overflow.
Com quem conversa?	top_final grava o resultado na BRAM.
Cuidado principal	o deslocamento para voltar de Q16.16 para Q8.8 é de 8 bits e há saturação.

Código completo:

```

module mac_unit (
    input wire      clk,
    input wire      rst_n,
    input wire      start,
    input wire      clear,
    input wire signed [15:0] operand_a_q88,
    input wire signed [15:0] operand_b_q88,
    output reg      done,
    output reg signed [31:0] accumulator_q1616,
    output reg signed [15:0] result_q88,
    output reg      overflow
);
    wire signed [31:0] product_q1616 = operand_a_q88 * operand_b_q88;
    wire signed [32:0] sum_ext =
        {accumulator_q1616[31], accumulator_q1616} +
        {product_q1616[31], product_q1616};

    reg signed [31:0] next_acc;
    reg next_overflow;
    reg signed [31:0] scaled_q88;

    always @(*) begin
        next_overflow = 1'b0;
        if (sum_ext > 33'sd2147483647) begin
            next_acc = 32'sh7fffffff;
            next_overflow = 1'b1;
        end else if (sum_ext < -33'sd2147483648) begin
            next_acc = 32'sh80000000;
            next_overflow = 1'b1;
        end else begin

```

```

    next_acc = sum_ext[31:0];
end
scaled_q88 = next_acc >>> 8;
end

function signed [15:0] saturate_q88;
input signed [31:0] value;
begin
    if (value > 32'sd32767)
        saturate_q88 = 16'sh7fff;
    else if (value < -32'sd32768)
        saturate_q88 = 16'sh8000;

    else
        saturate_q88 = value[15:0];
    end
end
endfunction

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        done <= 1'b0;
        accumulator_q1616 <= 32'sd0;
        result_q88 <= 16'sd0;
        overflow <= 1'b0;
    end else begin
        done <= 1'b0;
        if (clear) begin
            accumulator_q1616 <= 32'sd0;
            result_q88 <= 16'sd0;
            overflow <= 1'b0;
        end else if (start) begin
            accumulator_q1616 <= next_acc;
            result_q88 <= saturate_q88(scaled_q88);
            overflow <= overflow | next_overflow;
            done <= 1'b1;
        end
    end
end
end
endmodule

```

12.13 fpga/rtl/memory/telemetry_bram.v

Onde criar: final_project/fpga/rtl/memory/telemetry_bram.v

Analogia: É o caderno de 1024 linhas onde os resultados do MAC são anotados.

Pergunta	Resposta
O que entra?	write_en/address/data e read_addr.
O que sai?	read_data.
Com quem conversa?	MAC escreve; pattern_stream lê para desenhar o gráfico.
Cuidado principal	a intenção é inferir BSRAM. Confira o relatório de recursos.

Código completo:

```

module telemetry_bram #(
    parameter ADDR_W = 10

```

```

)(
    input wire          clk,
    input wire          write_en,
    input wire [ADDR_W-1:0] write_addr,
    input wire signed [15:0] write_data,
    input wire [ADDR_W-1:0] read_addr,
    output reg signed [15:0] read_data
);
(* syn_ramstyle = "block_ram", ram_style = "block" *)
reg signed [15:0] memory [0:(1<<ADDR_W)-1];

always @(posedge clk) begin
    if (write_en)
        memory[write_addr] <= write_data;
    read_data <= memory[read_addr];
end
endmodule

```

12.14 fpga/rtl/control/command_executor.v

Onde criar: final_project/fpga/rtl/control/command_executor.v

Analogia: É o gerente: recebe um comando já validado e decide qual máquina deve agir.

Pergunta	Resposta
O que entra?	cmd_valid/id/payload e frame_start.
O que sai?	modo/cor/ganho, sinais do MAC, clear e pulsos de status.
Com quem conversa?	usado tanto na co-simulação quanto no caminho físico dos botões.
Cuidado principal	modo e cor passam de pending para active somente em frame_start para evitar rasgo.

Código completo:

```

module command_executor (
    input wire          clk,
    input wire          rst_n,
    input wire          frame_start,
    input wire          cmd_valid,
    output wire         cmd_ready,
    input wire [7:0]     cmd_id,
    input wire [31:0]    cmd_payload,
    output reg          accepted_pulse,
    output reg          command_error_pulse,
    output reg          clear_pulse,
    output reg          mac_start,
    output reg signed [15:0] mac_operand_a,

    output reg signed [15:0] mac_operand_b,
    output reg [1:0]       active_mode,
    output reg [23:0]      active_color,
    output reg signed [15:0] gain_q88
);
localparam CMD_SET_MODE   = 8'h01;
localparam CMD_SET_COLOR = 8'h02;
localparam CMD_SET_GAIN  = 8'h03;

```

```

localparam CMD_MAC_STEP = 8'h04;
localparam CMD_CLEAR    = 8'h05;
localparam CMD_GET_STATUS = 8'h06;
localparam CMD_PING     = 8'h7f;

reg [1:0] pending_mode;
reg [23:0] pending_color;

assign cmd_ready = 1'b1;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        pending_mode <= 2'd0;
        active_mode <= 2'd0;
        pending_color <= 24'h204080;
        active_color <= 24'h204080;
        gain_q88 <= 16'sd256;
        accepted_pulse <= 1'b0;
        command_error_pulse <= 1'b0;
        clear_pulse <= 1'b0;
        mac_start <= 1'b0;
        mac_operand_a <= 16'sd0;
        mac_operand_b <= 16'sd0;
    end else begin
        accepted_pulse <= 1'b0;
        command_error_pulse <= 1'b0;
        clear_pulse <= 1'b0;
        mac_start <= 1'b0;

        if (frame_start) begin
            active_mode <= pending_mode;
            active_color <= pending_color;
        end

        if (cmd_valid && cmd_ready) begin
            case (cmd_id)
                CMD_SET_MODE: begin
                    if (cmd_payload[7:0] <= 8'd3) begin
                        pending_mode <= cmd_payload[1:0];
                        accepted_pulse <= 1'b1;
                    end else begin
                        command_error_pulse <= 1'b1;
                    end
                end
                CMD_SET_COLOR: begin
                    pending_color <= {
                        cmd_payload[7:0],
                        cmd_payload[15:8],
                        cmd_payload[23:16]
                    };
                    accepted_pulse <= 1'b1;
                end
                CMD_SET_GAIN: begin
                    gain_q88 <= cmd_payload[15:0];
                    accepted_pulse <= 1'b1;
                end
                CMD_MAC_STEP: begin
                    mac_operand_a <= cmd_payload[15:0];
                    mac_operand_b <= cmd_payload[31:16];
                    mac_start <= 1'b1;
                    accepted_pulse <= 1'b1;
                end
                CMD_CLEAR: begin
                    clear_pulse <= 1'b1;
                    accepted_pulse <= 1'b1;
            end case
        end
    end
end

```

```

        end
        CMD_GET_STATUS, CMD_PING: begin
            accepted_pulse <= 1'b1;
        end
        default: begin
            command_error_pulse <= 1'b1;
        end
    endcase
end
end
end
endmodule

```

12.15 fpga/rtl/control/uart_rx.v

Onde criar: final_project/fpga/rtl/control/uart_rx.v

Analogia: É o ouvido serial da FPGA.

Pergunta	Resposta
O que entra?	linha rx.
O que sai?	data_valid, data_byte e framing_error.
Com quem conversa?	protocol_system usa o byte recebido para alimentar packet_parser.
Cuidado principal	na simulação CLKS_PER_BIT é reduzido para acelerar; no hardware lógico o valor nominal é 219.

Código completo:

```

module uart_rx #(
    parameter integer CLKS_PER_BIT = 219
) (
    input wire    clk,
    input wire    rst_n,
    input wire    rx,
    output reg     data_valid,
    output reg [7:0] data_byte,
    output reg     framing_error
);
    localparam S_IDLE = 3'd0;
    localparam S_START = 3'd1;
    localparam S_DATA = 3'd2;
    localparam S_STOP = 3'd3;

    localparam COUNT_W = $clog2(CLKS_PER_BIT + 1);
    reg [2:0] state;
    reg [COUNT_W-1:0] clock_count;
    reg [2:0] bit_index;
    reg [7:0] shift;
    reg rx_meta;
    reg rx_sync;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            rx_meta <= 1'b1;
            rx_sync <= 1'b1;
        end else begin
            rx_meta <= rx;

```

```

    rx_sync <= rx_meta;
end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= S_IDLE;
        clock_count <= {COUNT_W{1'b0}};
        bit_index <= 3'd0;
        shift <= 8'd0;
        data_valid <= 1'b0;
        data_byte <= 8'd0;
        framing_error <= 1'b0;
    end else begin
        data_valid <= 1'b0;
        framing_error <= 1'b0;
        case (state)
            S_IDLE: begin
                clock_count <= {COUNT_W{1'b0}};
                if (!rx_sync)
                    state <= S_START;
            end
            S_START: begin
                if (clock_count == (CLKS_PER_BIT / 2) - 1) begin
                    clock_count <= {COUNT_W{1'b0}};
                    if (!rx_sync) begin

                        bit_index <= 3'd0;
                        state <= S_DATA;
                    end else begin
                        state <= S_IDLE;
                    end
                end else begin
                    clock_count <= clock_count + 1'b1;
                end
            end
            S_DATA: begin
                if (clock_count == CLKS_PER_BIT - 1) begin
                    clock_count <= {COUNT_W{1'b0}};
                    shift[bit_index] <= rx_sync;
                    if (bit_index == 3'd7)
                        state <= S_STOP;
                    else
                        bit_index <= bit_index + 1'b1;
                end else begin
                    clock_count <= clock_count + 1'b1;
                end
            end
            S_STOP: begin
                if (clock_count == CLKS_PER_BIT - 1) begin
                    clock_count <= {COUNT_W{1'b0}};
                    if (rx_sync) begin
                        data_byte <= shift;
                        data_valid <= 1'b1;
                    end else begin
                        framing_error <= 1'b1;
                    end
                    state <= S_IDLE;
                end else begin
                    clock_count <= clock_count + 1'b1;
                end
            end
            default: state <= S_IDLE;
        endcase
    end
end
end

```

```
end
endmodule
```

12.16 fpga/rtl/control/uart_tx.v

Onde criar: final_project/fpga/rtl/control/uart_tx.v

Analogia: É a boca serial da FPGA.

Pergunta	Resposta
O que entra?	data_valid e data_byte.
O que sai?	linha tx, ready e busy.
Com quem conversa?	response_builder entrega bytes de ACK/NACK.
Cuidado principal	transmite start + 8 bits LSB-first + stop.

Código completo:

```
module uart_tx #(
    parameter integer CLKS_PER_BIT = 219
)(
    input wire    clk,
    input wire    rst_n,
    input wire    data_valid,
    output wire   data_ready,
    input wire [7:0] data_byte,
    output reg    tx,
    output reg    busy
);
localparam COUNT_W = $clog2(CLKS_PER_BIT + 1);
reg [COUNT_W-1:0] clock_count;
reg [3:0] bit_index;
reg [9:0] frame;

assign data_ready = !busy;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        tx <= 1'b1;
        busy <= 1'b0;
        clock_count <= {COUNT_W{1'b0}};
        bit_index <= 4'd0;
        frame <= 10'h3ff;
    end else if (!busy) begin
        tx <= 1'b1;

        if (data_valid) begin
            frame <= {1'b1, data_byte, 1'b0};
            tx <= 1'b0;
            busy <= 1'b1;
            clock_count <= {COUNT_W{1'b0}};
            bit_index <= 4'd0;
        end
    end else if (clock_count == CLKS_PER_BIT - 1) begin
        clock_count <= {COUNT_W{1'b0}};
        if (bit_index == 4'd9) begin
            tx <= 1'b1;
            busy <= 1'b0;
        end else begin
```

```

        bit_index <= bit_index + 1'b1;
        tx <= frame[bit_index + 1'b1];
    end
end else begin
    clock_count <= clock_count + 1'b1;
end
end
endmodule

```

12.17 fpga/rtl/control/packet_parser.v

Onde criar: final_project/fpga/rtl/control/packet_parser.v

Analogia: É a recepção da fábrica: abre o envelope, verifica selo, versão, tamanho e sequência.

Pergunta	Resposta
O que entra?	bytes da UART.
O que sai?	cmd_valid/id/payload ou resposta de erro.
Com quem conversa?	fica entre uart_rx e command_executor.
Cuidado principal	pacote com CRC ruim ou sequência repetida não deve executar o comando.

Código completo:

```

module packet_parser (
    input wire    clk,
    input wire    rst_n,
    input wire    byte_valid,
    input wire [7:0] byte_data,
    output reg    cmd_valid,
    output reg [7:0] cmd_seq,
    output reg [7:0] cmd_id,
    output reg [7:0] cmd_length,
    output reg [31:0] cmd_payload,
    output reg    response_valid,
    output reg [7:0] response_seq,
    output reg [7:0] response_status,
    output reg [15:0] response_error_count,
    output reg    error_pulse,
    output reg [15:0] error_count
);
    localparam STATUS_ACK      = 8'd0;
    localparam STATUS_BAD_CRC  = 8'd1;
    localparam STATUS_DUP_SEQ  = 8'd2;
    localparam STATUS_BAD_CMD  = 8'd3;
    localparam STATUS_BAD_LEN  = 8'd4;

    reg receiving;
    reg [3:0] index;
    reg [7:0] crc;
    reg [7:0] version_reg;
    reg [7:0] seq_reg;
    reg [7:0] cmd_reg;
    reg [7:0] length_reg;
    reg [31:0] payload_reg;
    reg have_last_seq;
    reg [7:0] last_seq;

    function [7:0] crc8_next;
        input [7:0] old_crc;
    endfunction

```

```

input [7:0] data;
integer bit_number;
reg [7:0] work;
begin
    work = old_crc ^ data;
    for (bit_number = 0; bit_number < 8; bit_number = bit_number + 1)
        work = work[7] ? (work << 1) ^ 8'h07 : (work << 1);
    crc8_next = work;
end
endfunction

```

```

function command_is_valid;
input [7:0] value;
begin
    command_is_valid =
        value == 8'h01 || value == 8'h02 || value == 8'h03 ||
        value == 8'h04 || value == 8'h05 || value == 8'h06 ||
        value == 8'h7f;
end
endfunction

```

```

task reject_packet;
input [7:0] status_value;
begin
    response_valid <= 1'b1;
    response_seq <= seq_reg;
    response_status <= status_value;
    response_error_count <= error_count + 16'd1;
    error_count <= error_count + 16'd1;
    error_pulse <= 1'b1;
end
endtask

```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        receiving <= 1'b0;
        index <= 4'd0;
        crc <= 8'd0;
        version_reg <= 8'd0;
        seq_reg <= 8'd0;
        cmd_reg <= 8'd0;
        length_reg <= 8'd0;
        payload_reg <= 32'd0;
        have_last_seq <= 1'b0;
        last_seq <= 8'd0;
        cmd_valid <= 1'b0;
        cmd_seq <= 8'd0;
        cmd_id <= 8'd0;
        cmd_length <= 8'd0;
        cmd_payload <= 32'd0;
        response_valid <= 1'b0;
        response_seq <= 8'd0;
        response_status <= 8'd0;
        response_error_count <= 16'd0;
        error_pulse <= 1'b0;
        error_count <= 16'd0;
    end else begin
        cmd_valid <= 1'b0;
        response_valid <= 1'b0;
        error_pulse <= 1'b0;

        if (byte_valid) begin
            if (!receiving) begin
                if (byte_data == 8'hA5) begin
                    receiving <= 1'b1;

```

```

        index <= 4'd0;
        crc <= 8'd0;
        payload_reg <= 32'd0;
    end
end else if (index < 4'd8) begin
    crc <= crc8_next(crc, byte_data);
    case (index)
        4'd0: version_reg <= byte_data;
        4'd1: seq_reg <= byte_data;
        4'd2: cmd_reg <= byte_data;
        4'd3: length_reg <= byte_data;
        4'd4: payload_reg[7:0] <= byte_data;
        4'd5: payload_reg[15:8] <= byte_data;
        4'd6: payload_reg[23:16] <= byte_data;
        4'd7: payload_reg[31:24] <= byte_data;
        default: ;
    endcase
    index <= index + 1'b1;

    end else begin
        receiving <= 1'b0;
        response_seq <= seq_reg;
        if (byte_data != crc) begin
            reject_packet(STATUS_BAD_CRC);
        end else if (version_reg != 8'h01 || !command_is_valid(cmd_reg)) begin
            reject_packet(STATUS_BAD_CMD);
        end else if (length_reg > 8'd4) begin
            reject_packet(STATUS_BAD_LEN);
        end else if (have_last_seq && seq_reg == last_seq) begin
            reject_packet(STATUS_DUP_SEQ);
        end else begin
            cmd_valid <= 1'b1;
            cmd_seq <= seq_reg;
            cmd_id <= cmd_reg;
            cmd_length <= length_reg;
            cmd_payload <= payload_reg;
            response_valid <= 1'b1;
            response_status <= STATUS_ACK;
            response_error_count <= error_count;
            last_seq <= seq_reg;
            have_last_seq <= 1'b1;
        end
    end
end
end
end
endmodule

```

Leia o fluxo do parser nesta ordem:

19. Fica parado até encontrar 0xA5, que marca o começo do pacote.
20. Recebe os oito bytes intermediários, guardando versão, sequência, comando, length e payload enquanto atualiza o CRC.
21. Quando chega o décimo byte, compara o CRC recebido com o CRC calculado.
22. Depois valida versão/comando, length e sequência duplicada.
23. Se algo falhar, gera status de erro e incrementa error_count.
24. Se tudo estiver correto, levanta cmd_valid por um ciclo e entrega o comando ao executor.

12.18 fpga/rtl/control/response_builder.v

Onde criar: final_project/fpga/rtl/control/response_builder.v

Analogia: É o atendente que monta a resposta de oito bytes.

Pergunta	Resposta
O que entra?	status, sequência, modo e contador de erros.
O que sai?	bytes para uart_tx.
Com quem conversa?	protocol_system liga o builder ao transmissor.
Cuidado principal	se já estiver enviando e chegar outra resposta, dropped_response sinaliza o problema.

Código completo:

```
module response_builder (  
    input wire      clk,  
    input wire      rst_n,  
    input wire      response_valid,  
    input wire [7:0] response_seq,  
    input wire [7:0] response_status,  
    input wire [1:0] active_mode,  
    input wire [15:0] error_count,  
    output wire      tx_data_valid,  
    input wire      tx_data_ready,  
    output wire [7:0] tx_data,  
    output reg      dropped_response  
);  
    reg [7:0] packet [0:7];  
    reg [2:0] index;  
    reg sending;  
  
    function [7:0] crc8_next;  
        input [7:0] old_crc;  
        input [7:0] data;  
        integer bit_number;  
        reg [7:0] work;  
        begin  
            work = old_crc ^ data;  
            for (bit_number = 0; bit_number < 8; bit_number = bit_number + 1)  
                work = work[7] ? (work << 1) ^ 8'h07 : (work << 1);  
            crc8_next = work;  
        end  
    endfunction  
  
    function [7:0] response_crc;  
        input [7:0] sequence_value;  
        input [7:0] status_value;  
        input [7:0] mode_value;  
        input [15:0] errors_value;  
        reg [7:0] work;  
        begin  
            work = crc8_next(8'd0, 8'h01);  
            work = crc8_next(work, sequence_value);  
  
            work = crc8_next(work, status_value);  
            work = crc8_next(work, mode_value);  
            work = crc8_next(work, errors_value[7:0]);  
            work = crc8_next(work, errors_value[15:8]);  
            response_crc = work;  
        end  
    endfunction
```

```

assign tx_data_valid = sending;
assign tx_data = packet[index];

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        index <= 3'd0;
        sending <= 1'b0;
        dropped_response <= 1'b0;
    end else begin
        dropped_response <= 1'b0;
        if (response_valid) begin
            if (!sending) begin
                packet[0] <= 8'h5A;
                packet[1] <= 8'h01;
                packet[2] <= response_seq;
                packet[3] <= response_status;
                packet[4] <= {6'd0, active_mode};
                packet[5] <= error_count[7:0];
                packet[6] <= error_count[15:8];
                packet[7] <= response_crc(
                    response_seq, response_status, {6'd0, active_mode}, error_count
                );
                index <= 3'd0;
                sending <= 1'b1;
            end else begin
                dropped_response <= 1'b1;
            end
        end
    end

    if (sending && tx_data_ready) begin
        if (index == 3'd7)
            sending <= 1'b0;
        else
            index <= index + 1'b1;
        end
    end
end
endmodule

```

12.19 fpga/rtl/protocol_system.v

Onde criar: final_project/fpga/rtl/protocol_system.v

Analogia: É uma bancada montada só para a co-simulação do protocolo completo.

Pergunta	Resposta
O que entra?	uart_rx_in, clock, reset e frame_start.
O que sai?	uart_tx_out e sinais de debug/estado.
Com quem conversa?	instancia RX, parser, executor, MAC, response_builder, TX e LEDs.
Cuidado principal	não é o top físico da placa; o top físico é top_final.

Código completo:

```

module protocol_system #(
    parameter integer CLKS_PER_BIT = 219,
    parameter integer CLK_HZ = 25_200_000
)(
    input wire        clk,
    input wire        rst_n,

```

```

input wire      uart_rx_in,
output wire     uart_tx_out,
input wire      frame_start,
output wire [1:0] active_mode,
output wire [23:0] active_color,
output wire signed [15:0] gain_q88,
output wire signed [15:0] mac_result_q88,
output wire [15:0] error_count,
output wire [2:0] ext_led,
output wire     response_valid_debug,
output wire [7:0] response_status_debug
);
wire rx_byte_valid;
wire [7:0] rx_byte;
wire framing_error;

uart_rx #(CLKS_PER_BIT(CLKS_PER_BIT)) receiver (
.clk(clk), .rst_n(rst_n), .rx(uart_rx_in),
.data_valid(rx_byte_valid), .data_byte(rx_byte), .framing_error(framing_error)
);

wire cmd_valid;
wire [7:0] cmd_seq;
wire [7:0] cmd_id;
wire [7:0] cmd_length;
wire [31:0] cmd_payload;
wire parser_response_valid;
wire [7:0] response_seq;
wire [7:0] response_status;
wire [15:0] response_error_count;
wire parser_error_pulse;
packet_parser parser (
.clk(clk), .rst_n(rst_n), .byte_valid(rx_byte_valid), .byte_data(rx_byte),
.cmd_valid(cmd_valid), .cmd_seq(cmd_seq), .cmd_id(cmd_id),
.cmd_length(cmd_length), .cmd_payload(cmd_payload),
.response_valid(parser_response_valid), .response_seq(response_seq),
.response_status(response_status), .response_error_count(response_error_count),
.error_pulse(parser_error_pulse), .error_count(error_count)
);

wire cmd_ready;
wire accepted_pulse;
wire command_error_pulse;
wire clear_pulse;
wire mac_start;
wire signed [15:0] mac_operand_a;
wire signed [15:0] mac_operand_b;
command_executor executor (
.clk(clk), .rst_n(rst_n), .frame_start(frame_start),
.cmd_valid(cmd_valid), .cmd_ready(cmd_ready), .cmd_id(cmd_id),
.cmd_payload(cmd_payload), .accepted_pulse(accepted_pulse),
.command_error_pulse(command_error_pulse), .clear_pulse(clear_pulse),
.mac_start(mac_start), .mac_operand_a(mac_operand_a),
.mac_operand_b(mac_operand_b), .active_mode(active_mode),
.active_color(active_color), .gain_q88(gain_q88)
);

wire mac_done;
wire signed [31:0] accumulator_q1616;
wire mac_overflow;
mac_unit mac (
.clk(clk), .rst_n(rst_n), .start(mac_start), .clear(clear_pulse),
.operand_a_q88(mac_operand_a), .operand_b_q88(mac_operand_b),
.done(mac_done), .accumulator_q1616(accumulator_q1616),
.result_q88(mac_result_q88), .overflow(mac_overflow)
);

```

```

);

wire tx_data_valid;
wire tx_data_ready;
wire [7:0] tx_data;
wire dropped_response;
response_builder responses (
    .clk(clk), .rst_n(rst_n), .response_valid(parser_response_valid),
    .response_seq(response_seq), .response_status(response_status),
    .active_mode(active_mode), .error_count(response_error_count),
    .tx_data_valid(tx_data_valid), .tx_data_ready(tx_data_ready),
    .tx_data(tx_data), .dropped_response(dropped_response)
);
wire tx_busy;
uart_tx #(CLKS_PER_BIT(CLKS_PER_BIT)) transmitter (
    .clk(clk), .rst_n(rst_n), .data_valid(tx_data_valid),
    .data_ready(tx_data_ready), .data_byte(tx_data), .tx(uart_tx_out), .busy(tx_busy)
);

led_status #(CLK_HZ(CLK_HZ), .PULSE_MS(2)) leds (
    .clk(clk), .rst_n(rst_n), .active_mode(active_mode),
    .accepted_pulse(accepted_pulse),
    .error_pulse(parser_error_pulse | command_error_pulse | framing_error | dropped_response),
    .mac_overflow(mac_overflow), .clear_pulse(clear_pulse), .ext_led(ext_led)

);

assign response_valid_debug = parser_response_valid;
assign response_status_debug = response_status;
endmodule

```

12.20 fpga/rtl/video/pattern_stream.v

Onde criar: final_project/fpga/rtl/video/pattern_stream.v

Analogia: É o pintor que cria os pixels, um de cada vez.

Pergunta	Resposta
O que entra?	modo, cor base e amostra de telemetria.
O que sai?	stream de pixels + frame_start.
Com quem conversa?	svo_hdmi_final passa esse stream para o encoder SVO.
Cuidado principal	x e y só avançam quando valid e ready estão ativos.

Código completo:

```

module pattern_stream #(
    parameter H_VISIBLE = 640,
    parameter V_VISIBLE = 480
)()
    input wire        clk,
    input wire        rst_n,
    input wire [1:0]   mode,
    input wire [23:0]  base_color,
    input wire signed [15:0] telemetry_sample,
    input wire [9:0]   telemetry_count,
    output wire [9:0]  telemetry_addr,
    output wire        frame_start,

```

```

output wire      out_axis_tvalid,
input wire       out_axis_tready,
output reg [23:0] out_axis_tdata,
output wire [0:0] out_axis_tuser
);
reg [9:0] x;
reg [8:0] y;
reg [7:0] red;
reg [7:0] green;
reg [7:0] blue;
reg signed [17:0] graph_y;

assign out_axis_tvalid = rst_n;
assign out_axis_tuser[0] = (x == 0) && (y == 0);
assign frame_start = out_axis_tvalid && out_axis_tready && (x == 0) && (y == 0);
assign telemetry_addr = x;

always @(*) begin
    red = 8'd0;
    green = 8'd0;
    blue = 8'd0;
    graph_y = 18'sd240 - (telemetry_sample >>> 2);

    case (mode)
        2'd0: begin
            if (x < 10'd80) begin red=8'hff; green=8'hff; blue=8'hff; end
            else if (x < 10'd160) begin red=8'hff; green=8'hff; blue=8'h00; end
            else if (x < 10'd240) begin red=8'h00; green=8'hff; blue=8'hff; end
            else if (x < 10'd320) begin red=8'h00; green=8'hff; blue=8'h00; end
            else if (x < 10'd400) begin red=8'hff; green=8'h00; blue=8'hff; end
            else if (x < 10'd480) begin red=8'hff; green=8'h00; blue=8'h00; end
            else if (x < 10'd560) begin red=8'h00; green=8'h00; blue=8'hff; end
            else begin red=8'h10; green=8'h10; blue=8'h10; end
        end
        2'd1: begin
            if (x[5] ^ y[5]) begin
                red = base_color[23:16];
                green = base_color[15:8];
                blue = base_color[7:0];
            end else begin
                red = ~base_color[23:16];
                green = ~base_color[15:8];
                blue = ~base_color[7:0];
            end
        end
        2'd2: begin
            red = x[7:0];

            green = y[7:0];
            blue = x[7:0] + y[7:0];
        end
        default: begin
            red = 8'h08;
            green = 8'h10;
            blue = 8'h18;
            if (y == 9'd240 || x == 10'd0) begin
                red = 8'h50;
                green = 8'h50;
                blue = 8'h50;
            end
            if (x < telemetry_count && graph_y >= 18'sd2 && graph_y <= 18'sd477 &&
                y >= graph_y - 18'sd2 && y <= graph_y + 18'sd2) begin
                red = 8'h20;
                green = 8'hff;
                blue = 8'h40;
            end
        end
    endcase
end

```

```

        end
    end
endcase

// SVO espera a ordem blue, green, red.
out_axis_tdata = {blue, green, red};
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        x <= 10'd0;
        y <= 9'd0;
    end else if (out_axis_tvalid && out_axis_tready) begin
        if (x == H_VISIBLE - 1) begin
            x <= 10'd0;
            if (y == V_VISIBLE - 1)
                y <= 9'd0;
            else
                y <= y + 1'b1;
        end else begin
            x <= x + 1'b1;
        end
    end
end
endmodule

```

12.21 fpga/rtl/video/svo_hdmi_final.v

Onde criar: final_project/fpga/rtl/video/svo_hdmi_final.v

Analogia: É o adaptador entre o seu pintor e a 'caixa preta' HDMI oficial.

Pergunta	Resposta
O que entra?	pixels, clocks, modo, telemetria.
O que sai?	pares diferenciais TMDS.
Com quem conversa?	usa svo_enc, svo_tmnds, OSER10 e ELVDS_OBUF oficiais.
Cuidado principal	mantenha a pasta oficial src/hdmi e o include path; não copie só um arquivo isolado.

Código completo:

```

/*
 * Adaptacao do Simple Video Out usado no exemplo oficial da Sipeed.
 * O aviso de licenca dos arquivos SVO originais deve permanecer no projeto.
 */
`timescale 1ns / 1ps
`include "hdmi/svo_defines.vh"

module svo_hdmi_final (
    input wire      clk,
    input wire      resetn,
    input wire      clk_pixel,
    input wire      clk_5x_pixel,
    input wire      locked,
    input wire [1:0] mode,
    input wire [23:0] base_color,
    input wire signed [15:0] telemetry_sample,
    input wire [9:0] telemetry_count,
    output wire [9:0] telemetry_addr,
    output wire      frame_start,

```

```

output wire      tmds_clk_n,
output wire      tmds_clk_p,
output wire [2:0] tmds_d_n,
output wire [2:0] tmds_d_p
);
parameter SVO_MODE = "640x480V";

parameter SVO_FRAMERATE = 60;
parameter SVO_BITS_PER_PIXEL = 24;
parameter SVO_BITS_PER_RED = 8;
parameter SVO_BITS_PER_GREEN = 8;
parameter SVO_BITS_PER_BLUE = 8;
parameter SVO_BITS_PER_ALPHA = 0;

wire source_tvalid;
wire source_tready;
wire [SVO_BITS_PER_PIXEL-1:0] source_tdata;
wire [0:0] source_tuser;
wire video_enc_tvalid;
wire video_enc_tready;
wire [SVO_BITS_PER_PIXEL-1:0] video_enc_tdata;
wire [3:0] video_enc_tuser;
wire [2:0] tmds_d;
wire [2:0] tmds_d0, tmds_d1, tmds_d2, tmds_d3, tmds_d4;
wire [2:0] tmds_d5, tmds_d6, tmds_d7, tmds_d8, tmds_d9;
reg [3:0] resetn_clk_pixel_q;

always @(posedge clk_pixel) begin
    if (!resetn)
        resetn_clk_pixel_q <= 4'b0000;
    else
        resetn_clk_pixel_q <= {resetn_clk_pixel_q[2:0], 1'b1};
end
wire clk_pixel_resetn = locked && resetn_clk_pixel_q[3];

pattern_stream pattern_source (
    .clk(clk_pixel),
    .rst_n(clk_pixel_resetn),
    .mode(mode),
    .base_color(base_color),
    .telemetry_sample(telemetry_sample),
    .telemetry_count(telemetry_count),
    .telemetry_addr(telemetry_addr),
    .frame_start(frame_start),
    .out_axis_tvalid(source_tvalid),
    .out_axis_tready(source_tready),
    .out_axis_tdata(source_tdata),
    .out_axis_tuser(source_tuser)
);

svo_enc #( `SVO_PASS_PARAMS ) encoder (
    .clk(clk_pixel),
    .resetn(clk_pixel_resetn),
    .in_axis_tvalid(source_tvalid),
    .in_axis_tready(source_tready),
    .in_axis_tdata(source_tdata),
    .in_axis_tuser(source_tuser),
    .out_axis_tvalid(video_enc_tvalid),
    .out_axis_tready(video_enc_tready),
    .out_axis_tdata(video_enc_tdata),
    .out_axis_tuser(video_enc_tuser)
);
assign video_enc_tready = 1'b1;

svo_tmds tmds_channel_0 (

```

```

.clk(clk_pixel), .resetn(clk_pixel_resetn),
.de(!video_enc_tuser[3]), .ctrl(video_enc_tuser[2:1]),
.din(video_enc_tdata[23:16]),
.dout({tmds_d9[0],tmds_d8[0],tmds_d7[0],tmds_d6[0],tmds_d5[0],
      tmds_d4[0],tmds_d3[0],tmds_d2[0],tmds_d1[0],tmds_d0[0]})
);
svo_tmds tmds_channel_1 (
.clk(clk_pixel), .resetn(clk_pixel_resetn),
.de(!video_enc_tuser[3]), .ctrl(2'b00),
.din(video_enc_tdata[15:8]),
.dout({tmds_d9[1],tmds_d8[1],tmds_d7[1],tmds_d6[1],tmds_d5[1],
      tmds_d4[1],tmds_d3[1],tmds_d2[1],tmds_d1[1],tmds_d0[1]})
);
svo_tmds tmds_channel_2 (
.clk(clk_pixel), .resetn(clk_pixel_resetn),

.de(!video_enc_tuser[3]), .ctrl(2'b00),
.din(video_enc_tdata[7:0]),
.dout({tmds_d9[2],tmds_d8[2],tmds_d7[2],tmds_d6[2],tmds_d5[2],
      tmds_d4[2],tmds_d3[2],tmds_d2[2],tmds_d1[2],tmds_d0[2]})
);

OSER10 serializers [2:0] (
.Q(tmds_d), .D0(tmds_d0), .D1(tmds_d1), .D2(tmds_d2),
.D3(tmds_d3), .D4(tmds_d4), .D5(tmds_d5), .D6(tmds_d6),
.D7(tmds_d7), .D8(tmds_d8), .D9(tmds_d9),
.PCLK(clk_pixel), .FCLK(clk_5x_pixel), .RESET(~clk_pixel_resetn)
);

ELVDS_OBUF differential_outputs [3:0] (
.I({clk_pixel, tmds_d}),
.O({tmds_clk_p, tmds_d_p}),
.OB({tmds_clk_n, tmds_d_n})
);
endmodule

```

Não confunda: svo_hdmi_final.v é SEU adaptador. Os arquivos svo_enc.v, svo_tmds.v, svo_defines.vh e demais SVO continuam vindo da pasta oficial fpga/gowin/src/hdmi.

12.22 fpga/rtl/top_final.v

Onde criar: final_project/fpga/rtl/top_final.v

Analogia: É o maestro da placa. Todos os instrumentos físicos entram aqui.

Pergunta	Resposta
O que entra?	clock e dois botões.
O que sai?	LED onboard, LEDs externos e HDMI.
Com quem conversa?	instancia clocks, reset, botões, executor, MAC, BRAM, LEDs e vídeo.
Cuidado principal	é o único top que deve ser selecionado no Gowin para a implementação final.

Código completo:

```

module top_final (
    input wire    clk,
    input wire    usr_key_1,
    input wire    usr_key_2,
    output wire    led_pin,
    output wire [2:0] ext_led,
    output wire    tmds_clk_n,
    output wire    tmds_clk_p,
    output wire [2:0] tmds_d_n,
    output wire [2:0] tmds_d_p
);
    wire clk_126;
    wire clk_pixel;
    wire pll_lock;
    wire rst_n;

    Gowin_PLLVR pll (
        .clkout(clk_126),
        .lock(pll_lock),
        .clkin(clk)
    );
    Gowin_CLKDIV divider (
        .clkout(clk_pixel),
        .hclkin(clk_126),
        .resetn(pll_lock)
    );
    reset_sync reset_release (
        .clk(clk_pixel),
        .async_ready(pll_lock),
        .rst_n(rst_n)
    );

    wire key1_sync;
    wire key2_sync;
    wire key1_level;
    wire key2_level;
    wire key1_pulse;
    wire key2_pulse;
    sync2ff key1_synchronizer (
        .clk(clk_pixel), .rst_n(rst_n), .async_in(~usr_key_1), .sync_out(key1_sync)
    );
    sync2ff key2_synchronizer (
        .clk(clk_pixel), .rst_n(rst_n), .async_in(~usr_key_2), .sync_out(key2_sync)
    );
    debounce_pulse key1_debounce (
        .clk(clk_pixel), .rst_n(rst_n), .button_in(key1_sync),
        .button_level(key1_level), .pressed_pulse(key1_pulse)
    );

    debounce_pulse key2_debounce (
        .clk(clk_pixel), .rst_n(rst_n), .button_in(key2_sync),
        .button_level(key2_level), .pressed_pulse(key2_pulse)
    );

    wire local_cmd_valid;
    wire local_cmd_ready;
    wire [7:0] local_cmd_id;
    wire [31:0] local_cmd_payload;
    button_commands local_commands (
        .clk(clk_pixel), .rst_n(rst_n),
        .key1_level(key1_level), .key2_level(key2_level),
        .key1_pulse(key1_pulse), .key2_pulse(key2_pulse),
        .cmd_ready(local_cmd_ready), .cmd_valid(local_cmd_valid),
        .cmd_id(local_cmd_id), .cmd_payload(local_cmd_payload)
    );

```

```

wire frame_start;
wire accepted_pulse;
wire command_error_pulse;
wire clear_pulse;
wire mac_start;
wire signed [15:0] mac_operand_a;
wire signed [15:0] mac_operand_b;
wire [1:0] active_mode;
wire [23:0] active_color;
wire signed [15:0] gain_q88;
command_executor executor (
    .clk(clk_pixel), .rst_n(rst_n), .frame_start(frame_start),
    .cmd_valid(local_cmd_valid), .cmd_ready(local_cmd_ready),
    .cmd_id(local_cmd_id), .cmd_payload(local_cmd_payload),
    .accepted_pulse(accepted_pulse), .command_error_pulse(command_error_pulse),
    .clear_pulse(clear_pulse), .mac_start(mac_start),
    .mac_operand_a(mac_operand_a), .mac_operand_b(mac_operand_b),
    .active_mode(active_mode), .active_color(active_color), .gain_q88(gain_q88)
);

wire mac_done;
wire signed [31:0] accumulator_q1616;
wire signed [15:0] mac_result_q88;
wire mac_overflow;
mac_unit mac (
    .clk(clk_pixel), .rst_n(rst_n), .start(mac_start), .clear(clear_pulse),
    .operand_a_q88(mac_operand_a), .operand_b_q88(mac_operand_b),
    .done(mac_done), .accumulator_q1616(accumulator_q1616),
    .result_q88(mac_result_q88), .overflow(mac_overflow)
);

reg [9:0] telemetry_write_addr;
reg [9:0] telemetry_count;
wire [9:0] telemetry_read_addr;
wire signed [15:0] telemetry_read_data;
always @(posedge clk_pixel or negedge rst_n) begin
    if (!rst_n || clear_pulse) begin
        telemetry_write_addr <= 10'd0;
        telemetry_count <= 10'd0;
    end else if (mac_done) begin
        telemetry_write_addr <= telemetry_write_addr + 1'b1;
        if (telemetry_count != 10'h3ff)
            telemetry_count <= telemetry_count + 1'b1;
    end
end
telemetry_bram telemetry (
    .clk(clk_pixel), .write_en(mac_done),
    .write_addr(telemetry_write_addr), .write_data(mac_result_q88),
    .read_addr(telemetry_read_addr), .read_data(telemetry_read_data)
);

led_status leds (
    .clk(clk_pixel), .rst_n(rst_n), .active_mode(active_mode),
    .accepted_pulse(accepted_pulse), .error_pulse(command_error_pulse),
    .mac_overflow(mac_overflow), .clear_pulse(clear_pulse), .ext_led(ext_led)
);

svo_hdmi_final video (
    .clk(clk_pixel), .resetsn(rst_n),
    .clk_pixel(clk_pixel), .clk_5x_pixel(clk_126), .locked pll_lock,
    .mode(active_mode), .base_color(active_color),
    .telemetry_sample(telemetry_read_data), .telemetry_count(telemetry_count),
    .telemetry_addr(telemetry_read_addr), .frame_start(frame_start),

```

```

.tmds_clk_n(tmds_clk_n), .tmds_clk_p(tmds_clk_p),
.tmds_d_n(tmds_d_n), .tmds_d_p(tmds_d_p)
);

reg [24:0] heartbeat_count;
always @(posedge clk_pixel or negedge rst_n)
    if (!rst_n) heartbeat_count <= 25'd0;
    else heartbeat_count <= heartbeat_count + 1'b1;

// O LED onboard do Nano 4K e ativo em nível baixo.
assign led_pin = ~(heartbeat_count[24] & pll_lock);
endmodule

```

Leia top_final de cima para baixo como uma montagem:

25. Gowin_PLLVR cria clk_126 e pll_lock a partir dos 27 MHz da placa.
26. Gowin_CLKDIV divide o clock rápido e cria clk_pixel.
27. reset_sync só libera rst_n depois do lock do PLL.
28. sync2ff + debounce_pulse tratam KEY1 e KEY2.
29. button_commands converte os botões em cmd_valid/cmd_id/payload.
30. command_executor aplica os comandos.
31. mac_unit calcula; telemetry_bram guarda os resultados.
32. led_status dirige os três LEDs externos.
33. svo_hdmi_final gera HDMI usando os arquivos oficiais da Sipeed.
34. heartbeat_count pisca o LED onboard para mostrar que o clock continua vivo.

12.23 fpga/constraints/final.cst

Onde criar: final_project/fpga/constraints/final.cst

Analogia: É o mapa que diz em qual tomada física cada fio lógico chega.

Pergunta	Resposta
O que entra?	nomes de portas do top_final.
O que sai?	associação a pinos físicos.
Com quem conversa?	Gowin usa no Place and Route.
Cuidado principal	não deixe hdmi.cst ativo ao mesmo tempo; pinagem duplicada causa conflito.

Código completo:

```

IO_LOC "clk" 45;
IO_PORT "clk" IO_TYPE=LVCMS33;

IO_LOC "usr_key_1" 14;
IO_PORT "usr_key_1" PULL_MODE=UP BANK_VCCIO=1.8;
IO_LOC "usr_key_2" 15;
IO_PORT "usr_key_2" PULL_MODE=UP BANK_VCCIO=1.8;

IO_LOC "led_pin" 10;
IO_PORT "led_pin" IO_TYPE=LVCMS33 DRIVE=8 BANK_VCCIO=3.3;

IO_LOC "ext_led[0]" 39;
IO_PORT "ext_led[0]" IO_TYPE=LVCMS33 DRIVE=8 PULL_MODE=NONE;
IO_LOC "ext_led[1]" 40;
IO_PORT "ext_led[1]" IO_TYPE=LVCMS33 DRIVE=8 PULL_MODE=NONE;
IO_LOC "ext_led[2]" 41;

```

```
IO_PORT "ext_led[2]" IO_TYPE=LVC MOS33 DRIVE=8 PULL_MODE=NONE;

IO_LOC "tmds_d_p[0]" 30,29;
IO_PORT "tmds_d_p[0]" PULL_MODE=NONE DRIVE=8;
IO_LOC "tmds_d_p[1]" 32,31;
IO_PORT "tmds_d_p[1]" PULL_MODE=NONE DRIVE=8;
IO_LOC "tmds_d_p[2]" 35,34;
IO_PORT "tmds_d_p[2]" PULL_MODE=NONE DRIVE=8;
IO_LOC "tmds_clk_p" 28,27;
IO_PORT "tmds_clk_p" PULL_MODE=NONE DRIVE=8;
```

12.24 fpga/constraints/final.sdc

Onde criar: final_project/fpga/constraints/final.sdc

Analogia: É a ficha do metrônomo: informa que o clock de entrada tem período de 37,037 ns.

Pergunta	Resposta
O que entra?	porta clk.
O que sai?	restrição temporal para o analisador.
Com quem conversa?	Gowin usa na análise de timing.
Cuidado principal	27 MHz -> período aproximado 37,037 ns.

Código completo:

```
create_clock -name clk_osc -period 37.037 -waveform {0 18.518} [get_ports {clk}]
```

12.25 fpga/sim/tb_mac_unit.v

Onde criar: final_project/fpga/sim/tb_mac_unit.v

Analogia: Ensaio de bancada do MAC.

Pergunta	Resposta
O que entra?	estímulos artificiais.
O que sai?	PASS ou \$fatal e waveform.
Com quem conversa?	fpga/sim/Makefile compila esse teste isoladamente.
Cuidado principal	se falhar, não avance para integração.

Código completo:

```
`timescale 1ns/1ps
module tb_mac_unit;
    reg clk = 1'b0;
    reg rst_n = 1'b0;
    reg start = 1'b0;
    reg clear = 1'b0;
    reg signed [15:0] a = 16'sd0;
    reg signed [15:0] b = 16'sd0;
    wire done;
    wire signed [31:0] acc;
    wire signed [15:0] result;
    wire overflow;
```

```

integer failures = 0;

always #5 clk = ~clk;
mac_unit dut (
    .clk(clk), .rst_n(rst_n), .start(start), .clear(clear),
    .operand_a_q88(a), .operand_b_q88(b), .done(done),
    .accumulator_q1616(acc), .result_q88(result), .overflow(overflow)
);

task step;
    input signed [15:0] value_a;
    input signed [15:0] value_b;
    begin
        @(negedge clk); a = value_a; b = value_b; start = 1'b1;
        @(negedge clk); start = 1'b0;
        @(negedge clk);
    end
endtask

task expect_result;
    input signed [15:0] expected;
    begin
        if (result != expected) begin
            $display("FALHA result=%0d esperado=%0d", result, expected);
            failures = failures + 1;
        end
    end
endtask

initial begin
    $dumpfile("build/tb_mac_unit.vcd");
    $dumpvars(0, tb_mac_unit);
    repeat (4) @(negedge clk);
    rst_n = 1'b1;

    step(16'sd256, 16'sd256); // 1.0 x 1.0
    expect_result(16'sd256);
    step(-16'sd256, 16'sd128); // soma -0.5
    expect_result(16'sd128);

    @(negedge clk); clear = 1'b1;
    @(negedge clk); clear = 1'b0;
    @(negedge clk);
    expect_result(16'sd0);

    step(16'sd32767, 16'sd32767);
    step(16'sd32767, 16'sd32767);
    step(16'sd32767, 16'sd32767);
    if (!overflow) begin
        $display("FALHA overflow nao ativou");

        failures = failures + 1;
    end

    if (failures == 0)
        $display("PASS tb_mac_unit");
    else
        $fatal(1, "FALHA tb_mac_unit: %0d erros", failures);
    $finish;
end
endmodule

```

12.26 fpga/sim/tb_led_status.v

Onde criar: final_project/fpga/sim/tb_led_status.v

Analogia: Ensaio do painel de LEDs sem placa física.

Pergunta	Resposta
O que entra?	modo, accepted, error, clear.
O que sai?	verificação de ext_led.
Com quem conversa?	reduz o clock/pulso por parâmetros para a simulação ser rápida.
Cuidado principal	não confunda esse teste com led_bringup físico.

Código completo:

```
`timescale 1ns/1ps
module tb_led_status;
    reg clk = 1'b0;
    reg rst_n = 1'b0;
    reg [1:0] active_mode = 2'd0;
    reg accepted_pulse = 1'b0;
    reg error_pulse = 1'b0;
    reg mac_overflow = 1'b0;
    reg clear_pulse = 1'b0;
    wire [2:0] ext_led;
    integer failures = 0;

    always #5 clk = ~clk;
    led_status #(.CLK_HZ(1000), .PULSE_MS(4)) dut (
        .clk(clk), .rst_n(rst_n), .active_mode(active_mode),
        .accepted_pulse(accepted_pulse), .error_pulse(error_pulse),
        .mac_overflow(mac_overflow), .clear_pulse(clear_pulse), .ext_led(ext_led)
    );

    initial begin
        $dumpfile("build/tb_led_status.vcd");
        $dumpvars(0, tb_led_status);
        repeat (3) @(negedge clk);
        rst_n = 1'b1;

        active_mode = 2'b10;
        @(negedge clk);
        if (ext_led[1:0] != 2'b10) failures = failures + 1;

        accepted_pulse = 1'b1;
        @(negedge clk); accepted_pulse = 1'b0;
        if (!ext_led[2]) failures = failures + 1;
        repeat (6) @(negedge clk);
        if (ext_led[2]) failures = failures + 1;

        error_pulse = 1'b1;
        @(negedge clk); error_pulse = 1'b0;
        repeat (6) @(negedge clk);
        if (!ext_led[2]) failures = failures + 1;

        clear_pulse = 1'b1;
        @(negedge clk); clear_pulse = 1'b0;
        @(negedge clk);
        if (ext_led[2]) failures = failures + 1;

        if (failures == 0)
```

```

    $display("PASS tb_led_status");
else
    $fatal(1, "FALHA tb_led_status: %0d erros", failures);
$finish;
end
endmodule

```

12.27 fpga/sim/tb_pattern_stream.v

Onde criar: final_project/fpga/sim/tb_pattern_stream.v

Analogia: Ensaio do pintor de pixels.

Pergunta	Resposta
O que entra?	ready alternando para simular backpressure.
O que sai?	contagem de pixels aceitos e inícios de quadro.
Com quem conversa?	testa pattern_stream sem HDMI físico.
Cuidado principal	H_VISIBLE/V_VISIBLE são reduzidos para 8x4 durante o teste.

Código completo:

```

`timescale 1ns/1ps
module tb_pattern_stream;
    reg clk = 1'b0;
    reg rst_n = 1'b0;
    reg [1:0] mode = 2'd1;
    reg ready = 1'b1;
    wire [9:0] telemetry_addr;
    wire frame_start;
    wire valid;
    wire [23:0] data;
    wire [0:0] user;
    integer accepted = 0;
    integer starts = 0;

    always #5 clk = ~clk;
    pattern_stream #(H_VISIBLE(8), V_VISIBLE(4)) dut (
        .clk(clk), .rst_n(rst_n), .mode(mode), .base_color(24'h204080),
        .telemetry_sample(16'sd0), .telemetry_count(10'd0),
        .telemetry_addr(telemetry_addr), .frame_start(frame_start),
        .out_axis_tvalid(valid), .out_axis_tready(ready),
        .out_axis_tdata(data), .out_axis_tuser(user)
    );

    always @(posedge clk) begin
        if (valid && ready) begin
            accepted = accepted + 1;
            if (user[0]) starts = starts + 1;
        end
    end

    initial begin
        $dumpfile("build/tb_pattern_stream.vcd");
        $dumpvars(0, tb_pattern_stream);
        repeat (3) @(negedge clk);
        rst_n = 1'b1;
        repeat (10) @(negedge clk);
        ready = 1'b0;
        repeat (5) @(negedge clk);
    end
endmodule

```

```

ready = 1'b1;
repeat (60) @(negedge clk);
if (accepted < 64 || starts < 2)
    $fatal(1, "FALHA stream accepted=%0d starts=%0d", accepted, starts);
$display("PASS tb_pattern_stream accepted=%0d starts=%0d", accepted, starts);
$finish;
end
endmodule

```

12.28 fpga/sim/tb_arm_fpga_protocol.v

Onde criar: final_project/fpga/sim/tb_arm_fpga_protocol.v

Analogia: É o ensaio geral. Usa os bytes reais produzidos pelo QEMU e simula o fio UART bit a bit.

Pergunta	Resposta
O que entra?	commands.hex.
O que sai?	ACKs, NACK de CRC, NACK de duplicata e estados finais.
Com quem conversa?	fecha a integração ARM64 -> arquivo -> UART simulada -> parser -> executor.
Cuidado principal	commands.hex deve ter 80 linhas antes de rodar.

Código completo:

```

`timescale 1ns/1ps
module tb_arm_fpga_protocol;
    localparam CLKS_PER_BIT = 8;
    localparam COMMAND_BYTES = 80;
    reg clk = 1'b0;
    reg rst_n = 1'b0;
    reg uart_rx_line = 1'b1;
    reg frame_start = 1'b0;
    wire uart_tx_line;
    wire [1:0] active_mode;
    wire [23:0] active_color;
    wire signed [15:0] gain_q88;
    wire signed [15:0] mac_result_q88;
    wire [15:0] error_count;
    wire [2:0] ext_led;
    wire response_valid;
    wire [7:0] response_status;

    reg [7:0] command_mem [0:COMMAND_BYTES-1];
    integer ack_count = 0;
    integer bad_crc_count = 0;
    integer duplicate_count = 0;
    integer tx_byte_count = 0;
    integer failures = 0;

    always #5 clk = ~clk;
    protocol_system #(CLKS_PER_BIT(CLK_PER_BIT), .CLK_HZ(1000)) dut (
        .clk(clk), .rst_n(rst_n), .uart_rx_in(uart_rx_line),
        .uart_tx_out(uart_tx_line), .frame_start(frame_start),
        .active_mode(active_mode), .active_color(active_color),
        .gain_q88(gain_q88), .mac_result_q88(mac_result_q88),
        .error_count(error_count), .ext_led(ext_led),
        .response_valid_debug(response_valid),
        .response_status_debug(response_status)
    );

```

```

);

wire tx_monitor_valid;
wire [7:0] tx_monitor_byte;
wire tx_monitor_error;
uart_rx #(CLKS_PER_BIT(CLKS_PER_BIT)) tx_monitor (
    .clk(clk), .rst_n(rst_n), .rx(uart_tx_line),
    .data_valid(tx_monitor_valid), .data_byte(tx_monitor_byte),
    .framing_error(tx_monitor_error)
);

always @(posedge clk) begin
    if (response_valid) begin
        case (response_status)
            8'd0: ack_count = ack_count + 1;
            8'd1: bad_crc_count = bad_crc_count + 1;
            8'd2: duplicate_count = duplicate_count + 1;
            default: failures = failures + 1;
        endcase
    end
    if (tx_monitor_valid)
        tx_byte_count = tx_byte_count + 1;
    if (tx_monitor_error)
        failures = failures + 1;
end

task send_uart_byte;
    input [7:0] value;
    integer bit_number;
    begin
        @(negedge clk); uart_rx_line = 1'b0;
        repeat (CLKS_PER_BIT) @(negedge clk);
        for (bit_number = 0; bit_number < 8; bit_number = bit_number + 1) begin
            uart_rx_line = value[bit_number];
            repeat (CLKS_PER_BIT) @(negedge clk);
        end
        uart_rx_line = 1'b1;
        repeat (CLKS_PER_BIT) @(negedge clk);
    end
endtask

task send_packet;
    input integer base;
    input corrupt_crc;
    integer offset;
    reg [7:0] value;
    begin
        for (offset = 0; offset < 10; offset = offset + 1) begin
            value = command_mem[base + offset];
            if (corrupt_crc && offset == 9)
                value = value ^ 8'h01;
            send_uart_byte(value);
        end
    end
endtask

task apply_frame_start;

    begin
        @(negedge clk); frame_start = 1'b1;
        @(negedge clk); frame_start = 1'b0;
    end
endtask

```

```

initial begin
    $readmemh("commands.hex", command_mem);
    $dumpfile("build/tb_arm_fpga_protocol.vcd");
    $dumpvars(0, tb_arm_fpga_protocol);
    repeat (6) @(negedge clk);
    rst_n = 1'b1;

    send_packet(0, 1'b0); apply_frame_start();
    send_packet(10, 1'b0); apply_frame_start();
    send_packet(20, 1'b0); apply_frame_start();
    send_packet(30, 1'b0); apply_frame_start();
    send_packet(40, 1'b0); apply_frame_start();
    send_packet(50, 1'b0); apply_frame_start();
    send_packet(60, 1'b0); apply_frame_start();
    send_packet(70, 1'b0); apply_frame_start();
    repeat (20) @(negedge clk);

    if (active_mode !== 2'd3) begin
        $display("FALHA active_mode=%0d", active_mode);
        failures = failures + 1;
    end
    if (active_color !== 24'h2080e0) begin
        $display("FALHA active_color=%h", active_color);
        failures = failures + 1;
    end
    if (gain_q88 !== 16'sh0180) begin
        $display("FALHA gain=%0d", gain_q88);
        failures = failures + 1;
    end
    if (mac_result_q88 !== 16'sd64) begin
        $display("FALHA MAC=%0d", mac_result_q88);
        failures = failures + 1;
    end

    send_packet(0, 1'b1); // CRC incorreto
    send_packet(70, 1'b0); // sequencia 7 repetida
    repeat (1000) @(negedge clk);

    if (ack_count != 8) begin
        $display("FALHA ACK=%0d", ack_count);
        failures = failures + 1;
    end
    if (bad_crc_count != 1 || duplicate_count != 1 || error_count != 2) begin
        $display("FALHA NACK crc=%0d dup=%0d errors=%0d",
            bad_crc_count, duplicate_count, error_count);
        failures = failures + 1;
    end
    if (tx_byte_count < 80) begin
        $display("FALHA TX retornou apenas %0d bytes", tx_byte_count);
        failures = failures + 1;
    end

    if (failures == 0)
        $display("PASS integracao ARM64 FPGA: 8 ACK, CRC e duplicata detectados");
    else
        $fatal(1, "FALHA integracao: %0d erros", failures);
    $finish;
end
endmodule

```

12.29 fpga/sim/Makefile

Onde criar: final_project/fpga/sim/Makefile

Analogia: É o botão 'rodar todos os ensaios da FPGA'.

Pergunta	Resposta
O que entra?	RTL + testbenches + commands.hex.
O que sai?	executáveis .vvp, waveforms .vcd e mensagens PASS/FALHA.
Com quem conversa?	o Makefile raiz chama este arquivo.
Cuidado principal	protocol exige commands.hex; por isso o fluxo raiz executa ARM64 antes.

Código completo:

```
IVERILOG ?= iverilog
VVP ?= vvp

FLAGS := -g2012 -Wall
.RECIPEPREFIX := >
RTL := \
  ../rtl/control/uart_rx.v \
  ../rtl/control/uart_tx.v \
  ../rtl/control/packet_parser.v \
  ../rtl/control/response_builder.v \
  ../rtl/control/command_executor.v \
  ../rtl/dsp/mac_unit.v \
  ../rtl/io/led_status.v \
  ../rtl/protocol_system.v

.PHONY: all mac leds video protocol clean

all: mac leds video protocol

build:
>mkdir -p build

mac: build
>$(IVERILOG) $(FLAGS) -o build/tb_mac_unit.vvp ../rtl/dsp/mac_unit.v tb_mac_unit.v
>$(VVP) build/tb_mac_unit.vvp

leds: build
>$(IVERILOG) $(FLAGS) -o build/tb_led_status.vvp ../rtl/io/led_status.v tb_led_status.v
>$(VVP) build/tb_led_status.vvp

video: build
>$(IVERILOG) $(FLAGS) -o build/tb_pattern_stream.vvp ../rtl/video/pattern_stream.v tb_pattern_stream.v
>$(VVP) build/tb_pattern_stream.vvp

protocol: build
>@test -f commands.hex || (echo "FALHA: execute make -C ../arm64 hex primeiro"; exit 1)
>$(IVERILOG) $(FLAGS) -o build/tb_arm_fpga_protocol.vvp $(RTL) tb_arm_fpga_protocol.v
>$(VVP) build/tb_arm_fpga_protocol.vvp

clean:
>rm -rf build
```

12.30 Makefile

Onde criar: final_project/Makefile

Analogia: É o botão 'testar o projeto inteiro'.

Pergunta	Resposta
O que entra?	Makefile do ARM64 e Makefile da simulação.
O que sai?	fluxo completo com make all.
Com quem conversa?	executa arm64 check e depois fpga/sim all.
Cuidado principal	rode a partir da raiz final_project.

Código completo:

```
.PHONY: all arm64 sim clean
.RECIPEPREFIX := >

all: arm64 sim

arm64:
>$(MAKE) -C arm64 check

sim: arm64
>$(MAKE) -C fpga/sim all

clean:
>$(MAKE) -C arm64 clean
>$(MAKE) -C fpga/sim clean
>rm -f fpga/sim/commands.hex
```

13. Arquivos de documentação e evidência

Crie docs/protocol.md para registrar a regra dos pacotes:

```
# Protocolo ARM64 -> FPGA

Pacote de comando: 10 bytes.

| Índice | Campo | Conteúdo |
|---:|---|---|
| 0 | SOF | `0xA5` |
| 1 | Versão | `0x01` |
| 2 | Sequência | Incrementa por comando |
| 3 | Comando | ID da operação |
| 4 | Length | 0 a 4 |
| 5-8 | Payload | 4 bytes; zeros quando não usados |
| 9 | CRC-8 | Polinômio `0x07` sobre bytes 1 a 8 |

Comandos: `01 SET_MODE`, `02 SET_COLOR`, `03 SET_GAIN`, `04 MAC_STEP`,
`05 CLEAR`, `06 GET_STATUS`, `7F PING`.

Resposta: 8 bytes iniciando em `0x5A`, com versão, sequência, status,
modo ativo, contador de erros (LSB/MSB) e CRC.

CRC-8: poly `0x07`, init `0x00`, sem reflexão, xorout `0x00`.
```

Use as pastas de evidence assim:

Pasta	Exemplos
evidence/logs	qemu_tests.txt, saída do make all.
evidence/waveforms	waveform_packet_ack.png, waveform_crc_nack.png,

	waveform_mac_q88.png.
evidence/synthesis	resource_summary.png, timing_summary.png, pin_report.png.
evidence/hardware	led_chase.mp4, hdmi_leds.jpg, demo_final.mp4.

14. Execute o software ARM64 primeiro

No WSL:

```
cd /mnt/c/Projetos/vga-framebuffer-arm-fpga/final_project
make clean
make -C arm64 check
```

O que deve acontecer:

- main.s, packet.s e syscalls.s são montados e linkados.
- qemu-aarch64 executa packet_gen.
- commands.bin é criado com 80 bytes.
- bin_to_hex.py cria fpga/sim/commands.hex com 80 linhas.
- check_packets.py imprime oito pacotes PASS e 0 FALHA.

Confirme:

```
wc -c arm64/build/commands.bin
wc -l fpga/sim/commands.hex
head -20 fpga/sim/commands.hex
```

Resultado esperado: commands.bin = 80 bytes e commands.hex = 80 linhas.

15. Execute as simulações da FPGA

```
cd /mnt/c/Projetos/vga-framebuffer-arm-fpga/final_project
make -C fpga/sim all
```

Você deve ver mensagens equivalentes a:

```
PASS tb_mac_unit
PASS tb_led_status
PASS tb_pattern_stream accepted=70 starts=3
PASS integracao ARM64 FPGA: 8 ACK, CRC e duplicata detectados
```

Se quiser rodar tudo de uma vez:

```
cd /mnt/c/Projetos/vga-framebuffer-arm-fpga/final_project
make clean
make all
```

Abra a waveform principal:

```
gtkwave fpga/sim/build/tb_arm_fpga_protocol.vcd
```

16. Como ler a waveform sem se perder

35. Comece olhando uart_rx_line: é o 'fio' por onde os bits entram.
36. Depois olhe response_valid e response_status: status 0 é ACK, 1 é CRC inválido, 2 é sequência duplicada.
37. Adicione active_mode, active_color, gain_q88, mac_result_q88 e error_count.
38. No pacote válido, procure cmd_valid e confirme que o estado muda.
39. No CRC inválido, error_count deve subir e os registradores de controle devem permanecer inalterados.
40. Na sequência duplicada, o comando não deve ser executado de novo.

17. Montando o projeto final no Gowin - clique por clique

41. Abra o Gowin EDA no Windows.
42. File > Open > C:\Projetos\vga-framebuffer-arm-fpga\final_project\fpga\gowin\hdmi.gprj.
43. File > Save Project As > salve como final_project.gprj dentro da mesma pasta fpga/gowin.
44. Confirme o dispositivo GW1NSR-LV4CQN48PC6/I5.
45. Não remova src/hdmi, src/gowin_pllvr nem src/gowin_clkdiv.
46. Project > Add Files e adicione SOMENTE os arquivos físicos listados abaixo.
47. Clique com o botão direito em top_final.v > Set as Top Module.
48. Remova src/hdmi.cst e src/hdmi.sdc do PROJETO ATIVO; não precisa apagar do disco.
49. Project > Add Files > adicione final_project/fpga/constraints/final.cst e final.sdc.
50. Execute Synthesize.
51. Depois execute Place and Route.
52. Abra Resource/Utilization Report, Timing Analysis e Pin Report e salve prints.
53. Somente depois abra Tools > Programmer e programe primeiro em SRAM.

Arquivos autorais que entram no projeto físico Gowin:

```
fpga/rtl/common/reset_sync.v
fpga/rtl/common/sync2ff.v
fpga/rtl/io/debounce_pulse.v
fpga/rtl/io/button_commands.v
fpga/rtl/io/led_status.v
fpga/rtl/control/command_executor.v
fpga/rtl/dsp/mac_unit.v
fpga/rtl/memory/telemetry_bram.v
fpga/rtl/video/pattern_stream.v
fpga/rtl/video/svo_hdmi_final.v
fpga/rtl/top_final.v
```

Não adicione ao Gowin: protocol_system.v, uart_rx.v, uart_tx.v, packet_parser.v, response_builder.v e os testbenches são usados pela co-simulação. O top físico atual recebe comandos pelos botões.

18. O que conferir depois da síntese

Item	O que procurar	Se não aparecer
DSP	MULTADDALU18X18 ou equivalente >= 1	Confirme mac_unit no top e que o resultado alimenta BRAM/vídeo.
BSRAM	Pelo menos 1 bloco	Confirme telemetry_bram 1024x16;

		se necessário use o IP Core Generator.
PLLVR	Instância na hierarquia	Confira se a pasta oficial gowin_pllvr permaneceu no projeto.
CLKDIV	Instância na hierarquia	Confira src/gowin_clkdiv.
OSER10	Serializadores TMDS	Confira os SVO e svo_hdmi_final.
ELVDS_OBUF	Saídas diferenciais HDMI	Confira pipeline SVO e constraints.
Timing	Slack não negativo	Não programe a placa enquanto houver violação crítica não entendida.

19. Programação em SRAM e teste físico

54. Desligue a placa e monte os LEDs conforme a seção de protoboard.
55. Conecte USB-C ao PC e HDMI da Tang Nano ao monitor.
56. No Gowin, Tools > Programmer.
57. Scan Device e confirme que a FPGA foi encontrada.
58. Selecione SRAM Program.
59. Aponte para o arquivo .fs em fpga/gowin/impl/pnr.
60. Program/Configure.
61. Ao iniciar, o monitor deve mostrar barras de cores e o LED onboard deve apresentar heartbeat.

Teste os botões nesta ordem:

Ação	HDMI	LED1 LED0	LED2
Inicialização	Barras de cores	00	Apaga após atividade inicial
KEY1	Xadrez	01	Pulso de atividade
KEY1 novamente	Gradiente	10	Pulso de atividade
KEY1 novamente	Gráfico de telemetria	11	Pulso de atividade
KEY2	Nova amostra no gráfico	11	Pulso de atividade
KEY1 + KEY2	Limpa MAC/telemetria	mantém modo	limpa alerta

20. Diagnóstico: onde procurar quando algo der errado

Sintoma	Primeiro lugar para olhar	Raciocínio
HDMI oficial já não funciona	fpga/gowin original, cabo, monitor, PLL	Ainda não é problema do seu RTL.
Module not found Gowin_PLLVR	fpga/gowin/src/gowin_pllvr	A base oficial foi copiada

		incompleta ou removida do projeto.
svo_defines.vh not found	fpga/gowin/src/hdmi	O include oficial perdeu o caminho.
Pino duplicado	constraints ativas	hdmi.cst e final.cst estão ativos juntos.
make -C arm64 check falha	arm64/ e bridge/	Conserte antes de olhar Verilog.
commands.hex não existe	arm64 Makefile / bin_to_hex.py	Rode make -C arm64 check.
tb_mac_unit falha	mac_unit.v	Não investigue HDMI enquanto o MAC unitário falha.
LED físico não acende	bringup + montagem	Teste led_bringup antes de top_final.
DSP = 0	mac_unit / observabilidade	Multiplicação pode ter sido otimizada.
BSRAM = 0	telemetry_bram	Memória pode ter virado LUT/FF; considere IP de Block RAM.

21. O que é integração real neste projeto

O ARM64 executa no QEMU e gera bytes reais. Esses bytes viram commands.hex e são enviados bit a bit para a UART simulada no testbench. A Tang Nano física executa o mesmo tipo de executor por meio dos botões. Sem adaptador USB-UART externo, não diga que houve transporte elétrico ao vivo entre PC/QEMU e FPGA.

Frase segura para a apresentação: A integração ARM64-FPGA foi validada por co-simulação usando os bytes realmente produzidos pelo executável AArch64 no QEMU; a implementação física valida o datapath FPGA, HDMI, MAC, memória, botões e LEDs.

22. Ordem exata que eu recomendo seguir

62. Preparar WSL e conferir ferramentas.
63. Criar a árvore de pastas.
64. Clonar a Sipeed em third_party e copiar hdmi_720p para fpga/gowin.
65. Fazer o HDMI oficial funcionar sem alterações.
66. Montar e testar led_bringup.
67. Criar todos os arquivos autorais exatamente nos caminhos deste guia.
68. Rodar make -C arm64 check.
69. Confirmar 80 bytes e 80 linhas.
70. Rodar make -C fpga/sim all.
71. Abrir a waveform e salvar evidências.
72. Abrir a base oficial no Gowin e salvar como final_project.gprj.
73. Adicionar apenas os 11 arquivos físicos autorais.
74. Definir top_final como top.
75. Trocar constraints oficiais por final.cst/final.sdc.
76. Synthesize -> conferir DSP/BSRAM/IPs.
77. Place and Route -> conferir timing e pinos.

- 78. Programar em SRAM.
- 79. Testar KEY1, KEY2 e KEY1+KEY2.
- 80. Salvar fotos, vídeo, relatórios e logs.
- 81. Só depois montar o ZIP/relatório final.

23. Checklist final do aluno

- ☐ Sei apontar a diferença entre arm64, bridge, rtl, sim, constraints e gowin.
- ☐ Sei dizer quais arquivos eu criei e quais vieram da Sipeed.
- ☐ fpga/gowin contém a cópia completa de hdmi_720p.
- ☐ HDMI oficial funcionou antes da integração.
- ☐ led_bringup testou os três LEDs separadamente.
- ☐ make all termina sem FALHA.
- ☐ commands.bin tem 80 bytes e commands.hex tem 80 linhas.
- ☐ A integração mostra 8 ACKs, um erro de CRC e uma duplicata.
- ☐ top_final é o top selecionado no Gowin.
- ☐ Somente final.cst e final.sdc estão ativos como constraints finais.
- ☐ O relatório de recursos mostra DSP e BSRAM.
- ☐ PLLVR, CLKDIV, OSER10 e ELVDS_OBUF aparecem na implementação.
- ☐ KEY1 percorre os quatro modos.
- ☐ KEY2 executa MAC_STEP.
- ☐ KEY1 + KEY2 limpa o estado.
- ☐ Tenho prints, waveforms, foto da montagem e vídeo final.
- ☐ Meu relatório diferencia co-simulação de UART física.

24. Fontes e rastreabilidade

Esta versão foi reestruturada a partir do PDF 'Guia Completo de Implementação do Projeto Final ARM64 FPGA' fornecido pelo aluno. Os códigos autorais reproduzidos aqui vieram desse material. Para a base HDMI, o guia mantém a estratégia de clonar/copiar o projeto oficial, sem reescrever os arquivos de terceiros.

Base oficial consultada: <https://github.com/sipeed/TangNano-4K-example> — projeto hdmi_720p. A lista de arquivos preservados foi conferida no hdmi.gprj oficial em 26/09/2026.

Documentação da placa consultada: <https://wiki.sipeed.com/hardware/en/tang/Tang-Nano-4K/Nano-4K.html>